

A Project Report
on
**MTCNN TECHNOLOGY FOR REUNIFICATION WITH REWARD
OUTCOMES**

Submitted in partial fulfilment of the requirements for the award of Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

G. BHAVYA SREE
(22FE1A0564)

K. PURNIMA
(22FE1A0579)

M. ANU REKHA CHOWDARY
(22FE1A05B5)

K. JAYA KRISHNA
(22FE1A0585)

Under the guidance of

Dr. A. HANUMAIAH, Ph.D.

PROFESSOR

Department of Computer Science and Engineering



VIGNAN'S LARA
INSTITUTE OF TECHNOLOGY & SCIENCE
(AUTONOMOUS)

Approved by AICTE New Delhi & Affiliated to JNTUK Kakinada
Accredited by **NAAC 'A+' and NBA (CSE, IT, ECE, EEE & ME) | ISO 9001 : 2015**
Vadlamudi - 522 213, Guntur District

April – 2026



VIGNAN'S LARA

INSTITUTE OF TECHNOLOGY & SCIENCE

(AUTONOMOUS)

Approved by AICTE New Delhi & Affiliated to JNTUK Kakinada
Accredited by **NAAC 'A+' and NBA (CSE, IT, ECE, EEE & ME) | ISO 9001 : 2015**
Vadlamudi - 522 213, Guntur District

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled "**MTCNN TECHNOLOGY FOR REUNIFICATION WITH REWARD OUTCOMES**" is a bonafide work done by **G. BHAVYA SREE (22FE1A0564), K. PURNIMA (22FE1A0579), M. ANU REKHA CHOWDARY (22FE1A05B5), K. JAYA KRISHNA (22FE1A0585)** under my guidance and submitted in partial fulfilment for the award of the degree of **Bachelor of Technology** in **COMPUTER SCIENCE AND ENGINEERING** from **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA, KAKINADA**. The work embodied in this project report is not submitted to any University or Institution for the award of any Degree or Diploma.

Project Guide

Dr. A. Hanumaiah , Ph.D.

Professor

Head of the Department

Dr. K. Venkateswara Rao, Ph.D.

Professor

External Examiner

DECLARATION

We hereby declare that the project report entitled "**MTCNN TECHNOLOGY FOR REUNIFICATION WITH REWARD OUTCOMES**" is a record of an original work done by us under the guidance of **Dr. A. HANUMAIAH, Professor of Computer Science and Engineering** and this project report is submitted in the fulfilment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science and Engineering**. The results embodied in this project report are not submitted to any other University or Institute for the award of any Degree or Diploma.

PROJECT MEMBERS

SIGNATURE

G. BHAVYA SREE (22FE1A0564)

K. PURNIMA (22FE1A0579)

M. ANU REKHA CHOWDARY (22FE1A05B5)

K. JAYA KRISHNA (22FE1A0585)

Place: Vadlamudi.

Date:

ACKNOWLEDGEMENT

The satisfaction that accompanies with the successful completion of any task would be incomplete without the mention of people whose ceaseless cooperation made it possible, whose constant guidance and encouragement crown all efforts with success.

We are grateful to **Dr. A. HANUMAIAH**, professor, Department of Computer Science and Engineering for guiding through this project and for encouraging right from the beginning of the project till successful completion of the project. Every interaction with him was an inspiration.

We thank **Dr. K. VENKATESWARA RAO**, Professor & HOD, Department of Computer Science and Engineering for his support and valuable suggestions.

We also express our thanks to **Dr. K. PHANEENDRA KUMAR**, Principal, Vignan's Lara Institute of Technology & Science for providing the resources to carry out the project.

We also express our sincere thanks to our beloved **Chairman Dr. LAVU RATHAIAH** for providing support and stimulating environment for developing the project.

We also place our floral gratitude to all other teaching and lab technicians for their constant support and advice throughout the project.

Project Members

G. BHAVYA SREE	(22FE1A0564)
K. PURNIMA	(22FE1A0579)
M. ANUREKHACHOWDARY	(22FE1A05B5)
K. JAYA KRISHNA	(22FE1A0585)

Table of Contents

Description	Page No.
Abstract defined.i	Error! Bookmark not
List of Figures defined.ii	Error! Bookmark not
List of Tables defined.iii	Error! Bookmark not
List of Abbreviations defined.	Error! Bookmark not
 CHAPTER 1: INTRODUCTION	 1-5
1.1 Deep Learning	1-2
1.2 What is a Convolutional Neural Network	2-3
1.3 What is MTCNN	4
1.4 MTCNN Stages	5-6
1.4.1 Proposal Network (P-Net)	5
1.4.2 Refine Network (R-Net)	5
1.4.3 Output Network (O-Net)	5-6
1.5 Networks Used in MTCNN	6
CHAPTER 2: LITERATURE SURVEY	7-13
2.1 Journals	7-11
2.1.1 AI Based System for Missing Person Identification	7-8
2.1.2 KNN	8-9
2.1.3 SVM	9-10
2.1.4 CNN	10-11
2.1.5 Overcoming Limitations Using MTCNN	11
2.2 Existing System	11-12
2.3 Limitations of Existing Methods	12-13
CHAPTER 3: METHODOLOGY	14-34
3.1 Proposed System	14-15
3.1.1 Advantages of the Proposed System	15-15
3.2 Model Architecture Specifications	15-26
3.2.1 Input Data Collection	15-16

3.2.2 Image Preprocessing	16-17
3.2.3 Face Detection using MTCNN	17-18
3.2.4 Feature Extraction using Deep Learning	18-19
3.2.5 Database Management System	19-20
3.2.6 Matching and Similarity Calculation	20-22
3.2.7 Verification Process	22-24
3.2.8 Security Mechanisms	24-25
3.2.9 Additional Features	26
3.3 Proposed Architecture	26-31
3.3.1 Image Pre-processing	26
3.3.2 Face Detection (MTCNN)	26-27
3.3.3 Feature Extraction	28
3.3.4 Transformer Backbone	28-31
3.4 Features in Proposed System	32-33
3.4.1 Search by Filter	32
3.4.2 Reward System	32
3.4.3 Messaging System	32
3.4.4 Multi-Language Translator	32
3.4.5 Image / Face Recognition	32
3.4.6 NGO'S Support	33
3.5 System Requirements	33-34
CHAPTER 4: SYSTEM DESIGN	35-41
4.1 System Architecture	35-36
4.2 Flow Chart Diagram	36-37
4.3 UML Diagrams	38-41
4.3.1 Use Case Diagram	38
4.3.2 Class Diagram	39
4.3.3 Sequence Diagram	40
4.3.4 Activity Diagram	41

CHAPTER 5: SYSTEM TESTING	42-43
5.1 Purpose of Testing	42
5.2 Types of Tests	42
5.2.1 Unit Testing	42
5.2.2 Integration Testing	42
5.2.3 User Interface Testing	42
5.2.4 System Testing	43
5.2.5 Performance Testing	43
5.2.6 Security Testing	43
CHAPTER 6: RESULTS & DISCUSSIONS	44-49
6.1 Accuracy and Loss	44
6.2 Predicted Captions	45-49
CHAPTER 7: DEPLOYMENT	50-53
7.1 Introduction	50
7.2 Deployment Environment	50-51
7.3 Deployment Architecture	51
7.4 Deployment Process	51-52
7.5 Security Considerations	52-53
7.6 Maintenance and Updates	53
7.7 Summary	53
CHAPTER 8: CONCLUSION AND FUTURE WORK	54
CHAPTER 9: REFERENCES	55
APPENDIX	56-57

ABSTRACT

Thousands of people go missing daily worldwide, including children, elderly individuals with Alzheimer's, and others in distressing circumstances. Traditional search methods are resourceintensive and time-consuming, creating significant challenges for law enforcement agencies. This paper introduces an innovative AI-driven system utilizing Multi-Task Cascaded Convolutional Neural Networks (MTCNN) for facial detection and recognition to expedite missing person identification. The system features a centralized database where families and author-ities upload missing person images, while community members can upload suspected matches through an authorized portal. Enhanced with NGO support providing reward incentives for the top 50 prioritized cases, the system includes advanced features such as gender and location filtering, multilingual support, automated messaging, and real-time chat functionality. Our evaluation demonstrates significant improvements in resolution rates, increasing from 57% to 69% post-AI implementation in 2022, showcasing the system's effectiveness in reuniting missing individuals with their families while fostering community engagement through reward-based participation.

Keywords – MTCNN, Facial Recognition, Missing Person Identification, Deep Learning, CNN, NGO Support, Reward System, Community Engagement.

FIGURES

FIG NO.	FIGURE NAME	PAGE NO.
1.1	Architecture of Convolutional Neural Network	3
1.2	What is Multi-task Cascaded Convolutional Neural Network?	6
2.1	K-Nearest Neighbours	9
2.2	Support Vector Machine	10
2.3	Convolutional Neural Network	11
3.1	Image Preprocessing Pipeline	17
3.2	MTCNN Network Stages	18
3.4	Feature Extraction Process	19
3.5	Database Record Structure	20
3.6	Face Matching Process	22
3.7	Verification Process	23
3.8	Verification Parameters	24
3.9	An Architecture for Missing Person Identification System	27
3.10	MTCNN Face Detection Process	28
3.11	Feature Extraction Using Deep Learning Models	29

3.12	Different Stages in MTCNN	30
3.13	Deep Feature Generation and Centralized Data Repository	31
4.1	System Architecture	35
4.2	Flow Chart Diagram	37
4.3	Use Case Diagram	38
4.4	Class Diagram	39
4.5	Sequence Diagram	40
4.6	Activity Diagram	41
7.1	Home Page	45
7.2	Upload Missing Person Details	45
7.3	Missing Person Information	46
7.4	My Uploads	46
7.5	Search by Filter	47
7.6	Multi-Language Translator	47
7.7	NGO Support	48
7.8	Messenger (Text, Location, Documents/Pictures Sharing)	48
7.9	Match Finding	49
7.10	Generated Report	49

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
1	Types of input data	15-16
2	Feature Extraction Models	18
3	Similarity Metrics	21-22
4	Verification Parameters	23
5	Security Features	25

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ML	Machine Learning
CNN	Convolutional Neural Network
MTCNN	Multi-Task Cascaded Convolutional Neural Network
KNN	K-Nearest Neighbours
SVM	Support Vector Machine
LBP	Local Binary Pattern
IoT	Internet of Things
NGO	Non-Governmental Organization
P-Net	Proposal Network
R-Net	Refine Network
O-Net	Output Network
DBMS	Database Management System
UI	User Interface
API	Application Programming Interface
GPU	Graphics Processing Unit
CPU	Central Processing Unit

NMS

Non-Maximum Suppression

CHAPTER 1

INTRODUCTION

Every year, millions of individuals go missing across the world due to various circumstances such as natural disasters, accidents, human trafficking, Alzheimer's disease, and personal crises. Identifying and locating these missing persons is a major challenge for law enforcement agencies, social organizations, and families. Traditional search methods rely heavily on manual processes such as reviewing photographs, distributing missing person notices, and maintaining records in local databases. These approaches are often time-consuming, resource-intensive, and inefficient when dealing with large volumes of data. Furthermore, traditional systems face difficulties in handling complex situations such as age progression, changes in physical appearance, or poor quality images. With the rapid advancement of Artificial Intelligence and Machine Learning, modern technologies have emerged that can significantly improve the efficiency of missing person identification. Among these technologies, facial recognition systems based on deep learning have shown great potential. These systems are capable of analyzing unique facial features and comparing them with large datasets to identify individuals accurately. By automating the detection and recognition process, such technologies can reduce search time and improve the chances of successfully locating missing persons. The proposed system introduces an Artificial Intelligence driven approach for identifying missing individuals using facial recognition techniques. The system integrates advanced image processing algorithms with a centralized database to detect and match faces efficiently. A unique aspect of the system is the integration of community participation through NGO partnerships that provide reward incentives for the top prioritized missing person cases. This approach encourages public involvement and improves the chances of locating missing individuals more quickly. The system also includes additional features such as gender and location-based filtering, multilingual translation support, automated message generation, and chat functionality to ensure accessibility and usability across diverse communities.

1.1 Deep Learning

Missing Person Identification System, Deep Learning plays a crucial role in analyzing facial images and identifying missing individuals accurately. The system uses deep learning algorithms to automatically detect, extract, and compare facial features from images. Unlike

traditional image processing techniques that rely on manually designed features, deep learning models learn patterns directly from large datasets of facial images. This allows the system to recognize unique facial characteristics such as the shape of the eyes, nose, mouth, and overall facial structure. In this project, deep learning is mainly used for face detection and facial recognition. When an image of a missing person or a suspected match is uploaded to the system, the deep learning model processes the image and identifies whether a human face is present. For this purpose, the system uses a Multi-Task Cascaded Convolutional Neural Network (MTCNN), which is a deep learning model designed specifically for detecting faces in images. MTCNN accurately locates faces within an image and identifies important facial landmarks such as the eyes, nose, and mouth. These landmarks help the system align the face correctly before further processing. Once the face is detected, deep learning techniques are used to extract distinctive facial features from the image. These features are converted into numerical representations called feature vectors, which uniquely describe a person's facial structure. The system then compares these feature vectors with the stored feature vectors of missing persons in the database. If a high similarity is found between two feature vectors, the system identifies it as a potential match. Deep learning also helps the system handle various challenges commonly encountered in missing person identification. For example, it can recognize faces even when there are changes in age, facial expressions, lighting conditions, or image quality. The model is capable of learning robust facial patterns, which improves the accuracy of recognition even when the available images are low resolution or partially obscured. By integrating deep learning with a centralized database and facial recognition techniques, the system can automatically scan large numbers of images and quickly identify potential matches. This significantly reduces the time required to locate missing persons and improves the efficiency of search operations. Therefore, deep learning serves as the core technology that enables the proposed system to perform accurate, fast, and reliable missing person identification.

1.2 What is a Convolutional Neural Network?

A Convolutional Neural Network (CNN) is a type of deep learning algorithm that is mainly used for image processing and computer vision tasks. It is designed to automatically recognize patterns and features in images such as edges, shapes, textures, and objects. CNNs are inspired by the way the human brain processes visual information and are widely used in applications like face recognition, object detection, image classification, and medical image analysis.

A CNN works by passing an image through multiple layers of artificial neurons. The most important layer in a CNN is the convolutional layer, which applies filters to the input image to detect important visual features. These filters scan the image and capture patterns such as edges, corners, and textures. After the convolutional layer, a pooling layer is used to reduce the size of the feature maps and remove unnecessary information while keeping the most important features.

Finally, fully connected layers combine all the extracted features and make the final prediction or classification.

One of the main advantages of CNN is that it can automatically learn and extract important features from images without manual feature selection. As the network processes more data, it becomes better at recognizing patterns and identifying objects in images. Because of this ability, CNNs are widely used in facial recognition systems, self-driving cars, security surveillance, and biometric identification systems. In simple terms, a Convolutional Neural Network is a deep learning model that helps computers understand and analyze images by automatically detecting important visual features. In many modern systems, including facial recognition and missing person identification systems, CNN plays a crucial role in accurately analyzing images and identifying individuals.

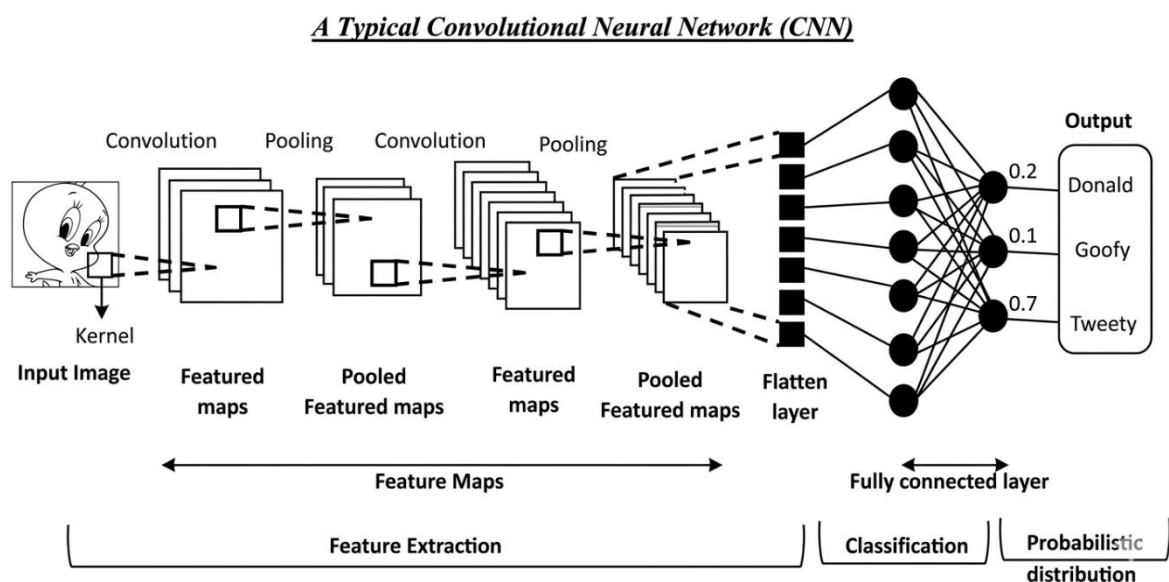


Fig 1.1 Architecture of Convolutional neural network

1.3 What is MTCNN?

Multi-Task Cascaded Convolutional Neural Network (MTCNN) is a deep learning framework used for face detection and facial landmark detection in images. It is widely used in computer vision applications such as facial recognition systems, surveillance systems, and missing person identification systems. MTCNN is designed to accurately detect human faces in images and locate important facial features such as the eyes, nose, and mouth.

MTCNN works by combining multiple Convolutional Neural Networks arranged in a cascade structure. Instead of detecting faces in a single step, the system processes the image through a sequence of neural networks that gradually improve the detection accuracy. This approach allows the model to quickly eliminate non-face regions and focus only on the areas that are likely to contain faces. As a result, MTCNN can detect faces even in challenging conditions such as lowresolution images, different lighting environments, varying facial expressions, and different face orientations.

The MTCNN framework performs multiple tasks simultaneously, including detecting faces, refining the detected face regions, and identifying facial landmarks. These landmarks help in aligning the face properly before performing further recognition tasks. Because of its ability to perform several related tasks at once, MTCNN provides more accurate and reliable results compared to traditional face detection methods. MTCNN consists of three neural networks that operate in sequence, namely the Proposal Network (P-Net), the Refine Network (R-Net), and the Output Network (O-Net). The P-Net scans the image and identifies possible face regions. The RNet then removes incorrect detections and improves the accuracy of the face bounding boxes. Finally, the O-Net performs the final face detection and identifies important facial landmarks such as the eyes, nose, and mouth.

In the proposed missing person identification system, MTCNN is used to detect faces from uploaded images before performing facial recognition. By accurately identifying and extracting the facial region, the system ensures that only relevant facial features are analyzed, which improves the overall accuracy and efficiency of the identification process.

1.4 MTCNN Stages

MTCNN (Multi-Task Cascaded Convolutional Neural Network) performs face detection through a three-stage cascaded process. Each stage uses a separate neural network that gradually improves the accuracy of face detection by filtering and refining the detected regions. These stages work sequentially to detect faces and identify facial landmarks in an image.

1.4.1 Proposal Network (P-Net)

In this stage, the input image is scanned at multiple scales to detect possible face regions. The P-Net quickly analyzes the image and generates several candidate bounding boxes where a face might be present. Since this stage processes the entire image, it focuses on speed and identifies potential face locations. To remove overlapping and incorrect detections, a technique called NonMaximum Suppression (NMS) is applied. This step eliminates many non-face regions and keeps only the most probable face candidates.

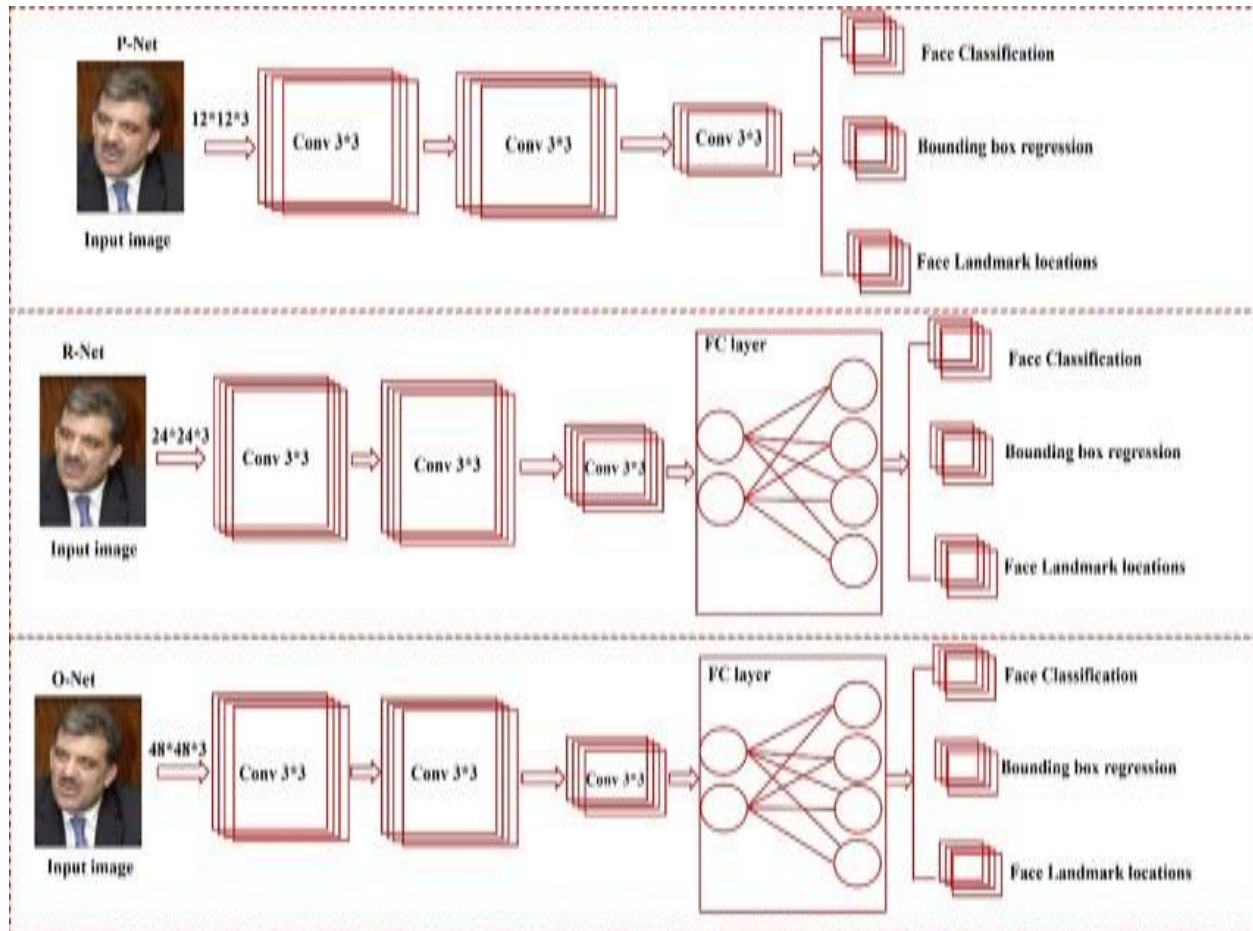
1.4.2 Refine Network (R-Net)

The candidate regions generated by the P-Net are passed to the R-Net for further analysis. This network performs a more detailed evaluation of each candidate region and removes false detections that do not contain faces. The R-Net also improves the accuracy of the bounding boxes around the detected faces by adjusting their positions and sizes. As a result, the number of candidate regions is reduced, and the remaining regions have a higher probability of containing actual faces.

1.4.3 Output Network (O-Net)

This is the final and most detailed stage of the MTCNN process. The O-Net performs precise face detection and further refines the bounding boxes. In addition to detecting faces, this network also identifies facial landmarks, such as the positions of the eyes, nose, and mouth. These landmarks are important because they help align the face correctly before performing facial recognition. The output of this stage provides the final detected face along with the coordinates of key facial features

Through these three stages P-Net, R-Net, and O-Net MTCNN ensures accurate and reliable face detection. Each stage progressively filters out incorrect detections and refines the face region, making the model highly effective for applications such as facial recognition, surveillance systems, and missing person identification systems.



1.5 Networks Used in MTCNN

Fig. 1.2 What is Multi-task Cascaded Convolutional Neural Network?

CHAPTER 2

LITERATURE SURVEY

2.1 JOURNALS

2.1.1 AI Based System for Missing Person Identification

The development of Artificial Intelligence (AI) based systems for missing person identification has gained significant attention in recent years. Researchers have explored various technologies including facial recognition, machine learning algorithms, Internet of Things (IoT), drone technology, and surveillance systems to improve the efficiency of locating missing individuals. Traditional search methods rely heavily on manual investigation and database comparison, which are time-consuming and inefficient when dealing with large amounts of data. With the rapid advancement of AI and deep learning techniques, automated systems have been developed to improve identification accuracy and reduce search time.

Kumar Singh et al. proposed a machine learning approach for missing person identification using the K-Nearest Neighbors (KNN) algorithm. In their system, images of missing individuals are uploaded into a database, and facial recognition models are used to compare facial encodings in order to identify potential matches. When a match is detected, the system automatically notifies law enforcement authorities and family members. Their work demonstrated the effectiveness of automated notification systems in improving the response time during missing person search operations.

Geetha et al. developed an automated system for identifying missing individuals that focuses on real-time processing capabilities. Their research emphasized the importance of quick response mechanisms in missing person cases. The system processes images and information rapidly, allowing authorities to take immediate action. This study established important principles for timecritical identification systems, which are further enhanced in modern AI-based solutions.

Facial recognition technology has significantly evolved with the introduction of Convolutional Neural Networks (CNNs). CNN-based models are highly effective in analyzing complex visual patterns and identifying facial features accurately. These models are widely used in security systems deployed in public places such as airports, railway stations, and surveillance centers CNNs have

shown strong performance in identifying faces even under varying environmental conditions, making them suitable for real-world missing person identification scenarios. Several traditional face detection techniques have also been used in earlier systems. The HAAR Cascade Classifier is one such method that performs real-time face detection by analyzing pixel intensity differences within an image. This method is commonly used in video surveillance applications. Similarly, Local Binary Pattern (LBP) based algorithms perform face recognition by extracting features based on pixel intensity relationships. LBP techniques are effective in handling variations in lighting conditions and facial expressions, which are common challenges in realworld facial recognition applications.

Despite these advancements, AI-based missing person identification systems face several challenges. Ethical and privacy concerns arise when handling sensitive personal data. Researchers have pointed out that improper data management may lead to misuse of personal information. Another major challenge is algorithmic bias, where facial recognition systems may produce higher error rates for certain demographic groups. Addressing these issues requires improved datasets, better training methods, and secure data handling practices. Overall, the integration of AI, machine learning, IoT, and surveillance technologies provides a promising solution for efficient missing person identification. However, further research is required to improve system accuracy, reduce false detection rates, and ensure secure and ethical use of personal data.

2.1.2 K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a simple and widely used machine learning algorithm that is mainly applied for classification tasks. In facial recognition systems, KNN works by comparing the features of a newly uploaded image with the features of images already stored in the database. The algorithm calculates the distance between the new image and the stored images and identifies the closest matches.

The image is then classified based on the majority class of its nearest neighbors. KNN is easy to implement and does not require complex training processes. However, it becomes inefficient when working with large datasets because the algorithm needs to compare the new image with all stored images. This increases computational time and reduces system performance when the database grows.

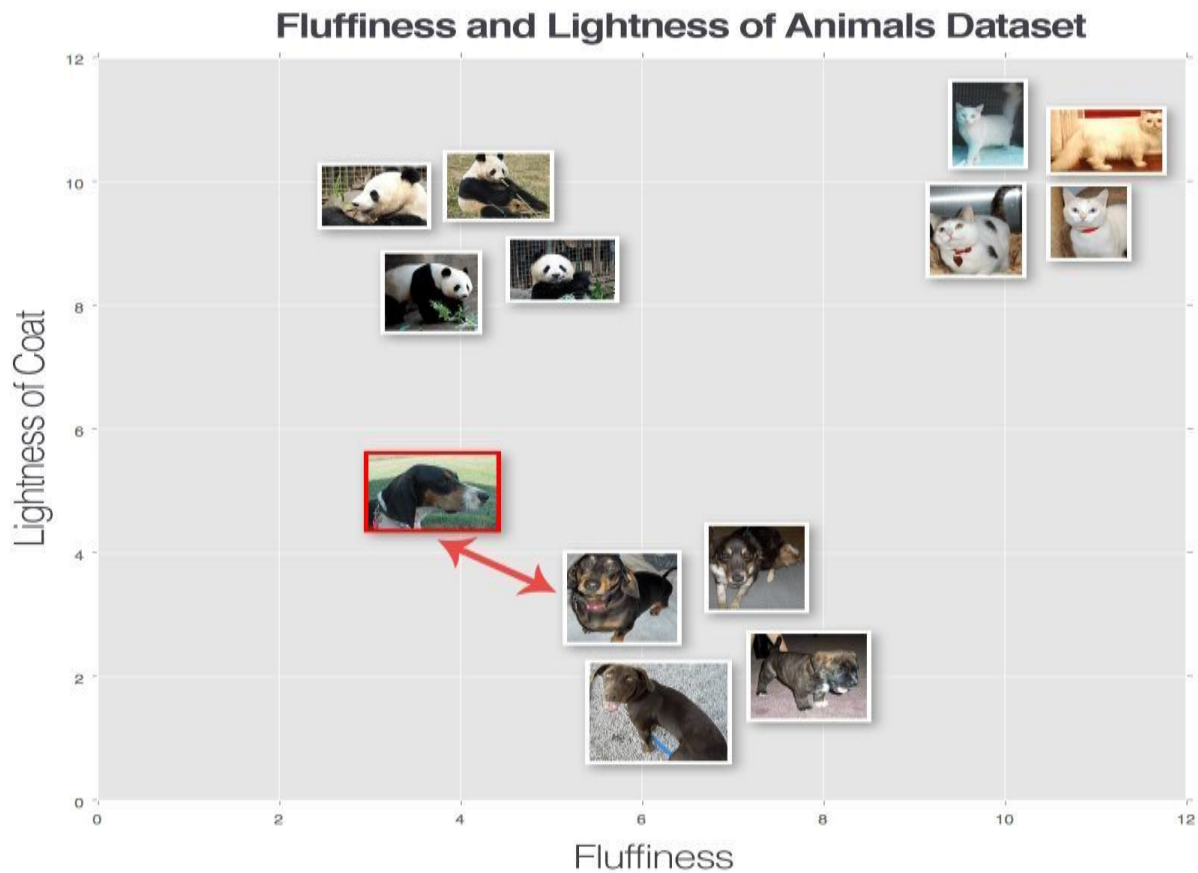


Fig 2.1: K-Nearest Neighbour's

2.1.3 Support Vector Machine (SVM)

Support Vector Machine (SVM) is another powerful machine learning algorithm commonly used for classification problems. SVM works by finding the optimal boundary, also known as a hyperplane, that separates different classes of data. The algorithm tries to maximize the margin between different classes to improve classification accuracy.

In facial recognition systems, SVM is used to classify facial features extracted from images. After extracting features such as facial landmarks or embeddings, the SVM classifier determines which class or person the face belongs to. Although SVM can achieve good classification accuracy, it requires carefully selected features and proper parameter tuning. Additionally, SVM may struggle when dealing with large-scale image datasets or complex variations in facial appearance such as lighting changes, facial expressions, or occlusions.

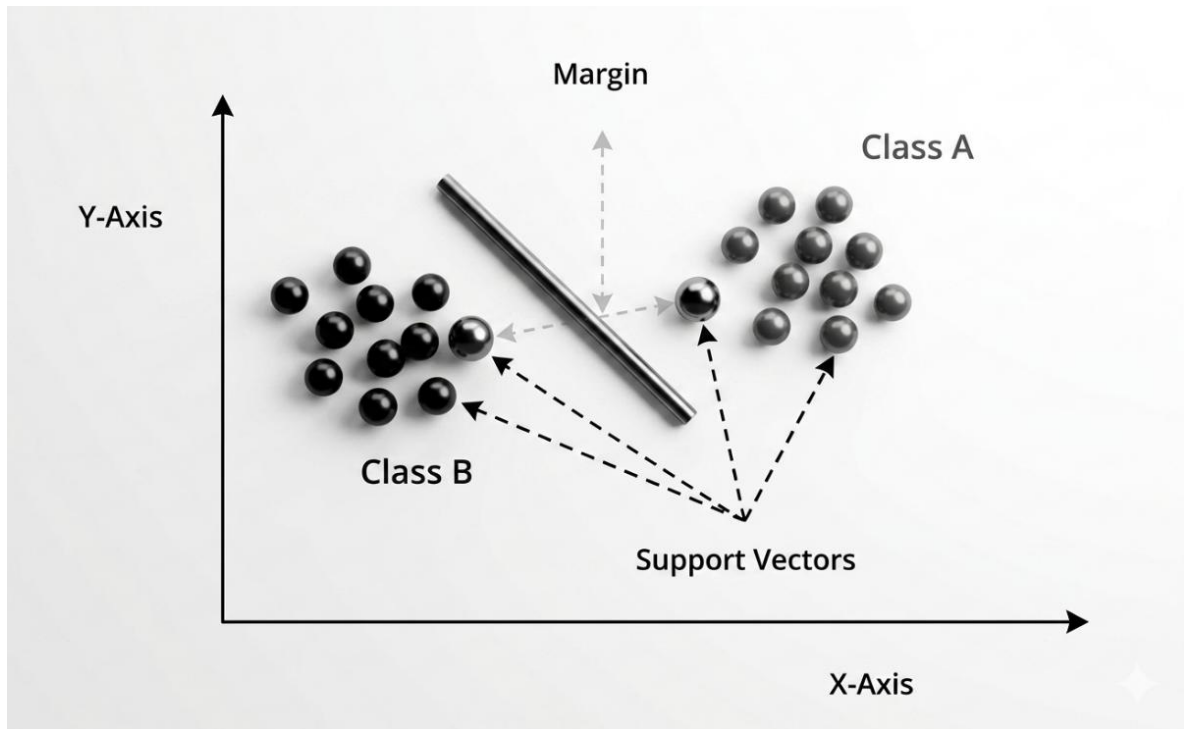


Fig 2.2: Support Vector Machine

2.1.4 Convolutional Neural Networks (CNN)

Convolutional Neural Networks (CNNs) are a type of deep learning model that has greatly improved the performance of image recognition systems. CNNs are designed to automatically learn and extract important visual features from images through multiple layers of convolution, pooling, and fully connected networks.

In facial recognition systems, CNNs analyze images and automatically detect important facial features such as eyes, nose, and mouth patterns. These features are then used to identify or verify a person's identity. Compared to traditional machine learning algorithms like KNN and SVM, CNNs provide higher accuracy and better performance in handling complex image variations.

However, CNN-based systems usually require a separate face detection method before performing face recognition. This additional step may increase the computational requirements and reduce efficiency in real-time applications if not optimized properly.

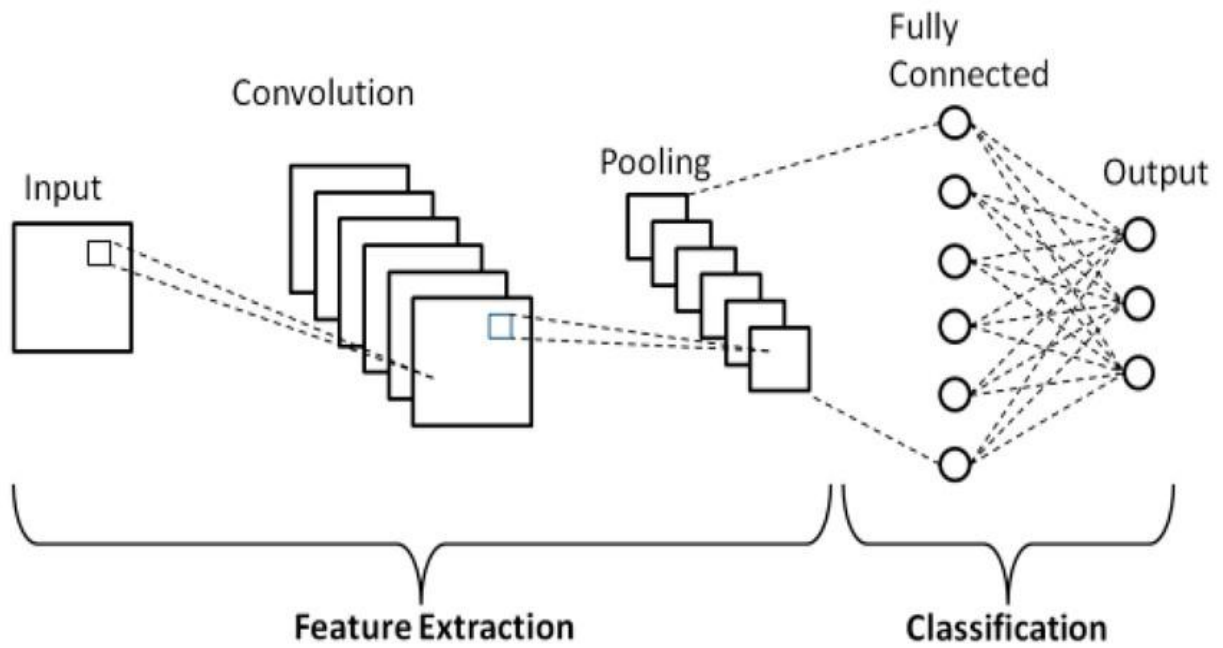


Fig 2.3: Convolutional Neural Network

2.1.5 Overcoming Limitations Using MTCNN

To overcome the limitations of traditional algorithms such as KNN, SVM, and basic CNN models, the proposed system uses Multi-Task Cascaded Convolutional Neural Networks (MTCNN) for face detection and facial landmark identification. MTCNN integrates multiple deep learning networks in a cascaded structure to detect faces more accurately. It performs multiple tasks simultaneously, including face detection, bounding box refinement, and facial landmark localization. Unlike traditional CNN models that require separate detection algorithms, MTCNN performs detection and alignment in a single framework.

The MTCNN architecture consists of three stages: Proposal Network (P-Net), Refine Network (R-Net), and Output Network (O-Net). These networks work sequentially to detect and refine face regions while eliminating false detections. The system also identifies key facial landmarks such as the eyes, nose, and mouth, which helps align faces correctly before recognition.

Compared to KNN and SVM, MTCNN provides significantly higher accuracy and faster detection performance. It can also detect faces in images with low resolution, different lighting conditions, and varying facial orientations. These advantages make MTCNN highly suitable for real-world missing person identification systems where image quality and environmental conditions cannot always be controlled.

2.2 Existing System

Traditional missing person identification systems have evolved over time with the advancement of technology. However, many systems still rely on conventional methods that limit their efficiency and scalability. In earlier approaches, the process of locating missing persons primarily depended on manual efforts carried out by law enforcement agencies and community members. These methods included distributing posters containing photographs and personal descriptions of missing individuals, conducting manual searches in public areas, reviewing tips received from the public, and collecting information through local police networks. While these methods have helped in several cases, they are often time-consuming and require significant human resources. When dealing with a large number of missing person cases or wide geographical areas, these traditional methods become inefficient and slow, which may delay rescue or identification efforts.

With the development of artificial intelligence and facial recognition technology, more advanced systems have been introduced to assist in identifying missing persons. Modern AI-based systems are capable of analyzing facial images and comparing them with images stored in centralized databases. These systems use deep learning algorithms to extract unique facial features and convert them into numerical representations that can be matched against large datasets. One notable example is the Face-Net system developed by Google, which uses deep learning algorithms to generate numerical feature vectors from facial images. These feature vectors represent unique facial characteristics and can be compared with other stored vectors to determine if two images belong to the same individual. This method allows rapid and efficient identification by searching through large databases in a short period of time.

Another well-known system is Clearview AI, which collects publicly available images from social media platforms and websites to create a large facial recognition database. Law enforcement agencies can upload an image of a person and search for similar faces within the database. This approach can accelerate the search process and help authorities identify missing individuals more quickly by comparing them with millions of images collected from various online sources. Therefore, there is a need for more advanced systems that combine accurate facial recognition with better community engagement and privacy protection mechanisms.

2.3 Limitations of Existing Methods

Despite the improvements introduced by AI-based facial recognition systems, several

Despite the improvements introduced by AI-based facial recognition systems, several

introduced by AI-based facial recognition systems, several limitations still exist in current missing person identification methods. One major limitation is the difficulty in recognizing faces that have undergone significant appearance changes. Factors such as aging, growth of facial hair, hairstyle changes, or other physical alterations can affect facial recognition accuracy. Many existing systems struggle to correctly match images when such variations occur.

Another challenge is the dependency on high-quality images. Facial recognition algorithms often fail when images are captured under poor lighting conditions, low resolution, or unusual camera angles. In real-world scenarios, images of missing persons may be old or unclear, which reduces the effectiveness of the recognition system. Privacy and ethical concerns also present significant challenges. Systems such as Clearview AI collect images from social media and other public sources without explicit user consent. This raises serious issues related to personal privacy, data protection, and ethical use of facial recognition technology.

Existing systems also lack strong mechanisms for community engagement. Public participation can play a crucial role in identifying missing persons, but most current systems do not provide incentives or structured platforms that encourage people to contribute information. In addition, many systems provide limited filtering options and search capabilities. For example, they may not support filtering based on gender, location, or other identifying information, which could help narrow down search results more efficiently.

Finally, existing systems often lack integrated communication features that allow coordination between users, authorities, and social organizations. Without proper communication channels, sharing information and responding quickly to potential matches becomes more difficult.

CHAPTER 3

METHODOLOGY

3.1 Proposed System

The proposed system leverages artificial intelligence and community collaboration to identify missing persons more effectively than traditional manual methods. It collects data from police records, social media, public reports, and crowdsourced submissions, then preprocesses it through image enhancement, face detection (MTCNN), and deduplication to ensure data quality.

At its core, deep learning models (FaceNet/VGG-Face) extract facial feature vectors from images, which are stored in a secure NoSQL database (MongoDB/Elasticsearch) with metadata like age, gender, and location. When a query image is submitted, the system compares it against stored vectors using cosine similarity or Euclidean distance, ranks matches by confidence, and generates detailed reports. Uncertain matches are flagged for manual review by authorities.

Additional features include NGO-backed reward incentives for public participation, gender/location-based filtering, multilingual support, real-time chat, and automated notifications — making it a scalable, accurate, and collaborative platform for missing person recovery.

3.1.1 Advantages of Proposed System

1. **Higher Identification Accuracy:** The use of deep learning models and MTCNN-based facial detection improves the accuracy of identifying missing persons even in challenging conditions.
2. **Efficient Data Processing:** The image preprocessing duplicate removal, and standardized
3. formats ensure better data quality and faster processing.
4. **Scalable Architecture:** The centralized NoSQL database system can handle large volumes of data, making the system suitable for national or large-scale implementation.
5. **Robust Matching Mechanism:** Similarity metrics such as cosine similarity and Euclidean distance enable reliable comparison between query images and stored records.
6. **Enhanced Security and Privacy:** Encryption and access control mechanisms protect
7. sensitive personal information stored in the database.
8. **Community Participation:** The NGO-supported reward system encourages public
9. involvement and increases the chances of locating missing persons.
10. **Advanced Search Features:** Gender and location-based filtering allow more targeted and

efficient searches.

11. **Improved Communication:** Integrated chat and automated messaging systems facilitate quick information sharing among families, law enforcement, and community members.
12. **Multilingual Accessibility:** Translation support makes the system usable for people from different linguistic background.
13. **Real-World Reliability:** The system can handle challenges such as aging effects, poor image quality, and partial facial visibility.

3.2 Model Architecture Specifications

3.2.1 Input Data Collection

The system begins with collecting facial images and related information from multiple reliable sources. These sources include police databases, missing person reports submitted by families, public submissions, NGO databases, and social media platforms. Gathering data from multiple channels helps build a diverse dataset that increases the probability of identifying missing individuals. Each record may include images, name, age, gender, last known location, and other identifying details.

This collected data is stored temporarily before preprocessing. The system must handle large volumes of structured and unstructured data, making it essential to maintain proper organization and indexing. Images can vary in resolution, lighting conditions, facial expressions, and angles. Therefore, the system prepares the dataset carefully before further processing to ensure reliable facial recognition performance.

Input Data Type	Description
Facial Images	Photos for detection and recognition.
Personal Details	Name, age, and gender.

Location	Last known seen location.
Police Records	Official law enforcement databases.
NGO/Public	Submissions from organizations and citizens.
Social Media	Information gathered from online platforms.

Table 1: Types of input data

3.2.2 Image Preprocessing

Image preprocessing improves the quality and consistency of images before they are used in facial recognition. Raw images may contain noise, low resolution, or irrelevant background information. To address these issues, preprocessing techniques such as image resizing, normalization, super-resolution enhancement, and duplicate removal are applied. These operations ensure that the input images meet the requirements of the facial detection model.

Another important preprocessing step is image normalization, which standardizes pixel values to improve neural network performance. Normalization scales pixel values to a range between 0 and 1, making training and feature extraction more stable. Preprocessing also reduces computational complexity and improves recognition accuracy.

Normalization Formula :

$$X_{norm} = (X - X_{min}) / (X_{max} - X_{min})$$

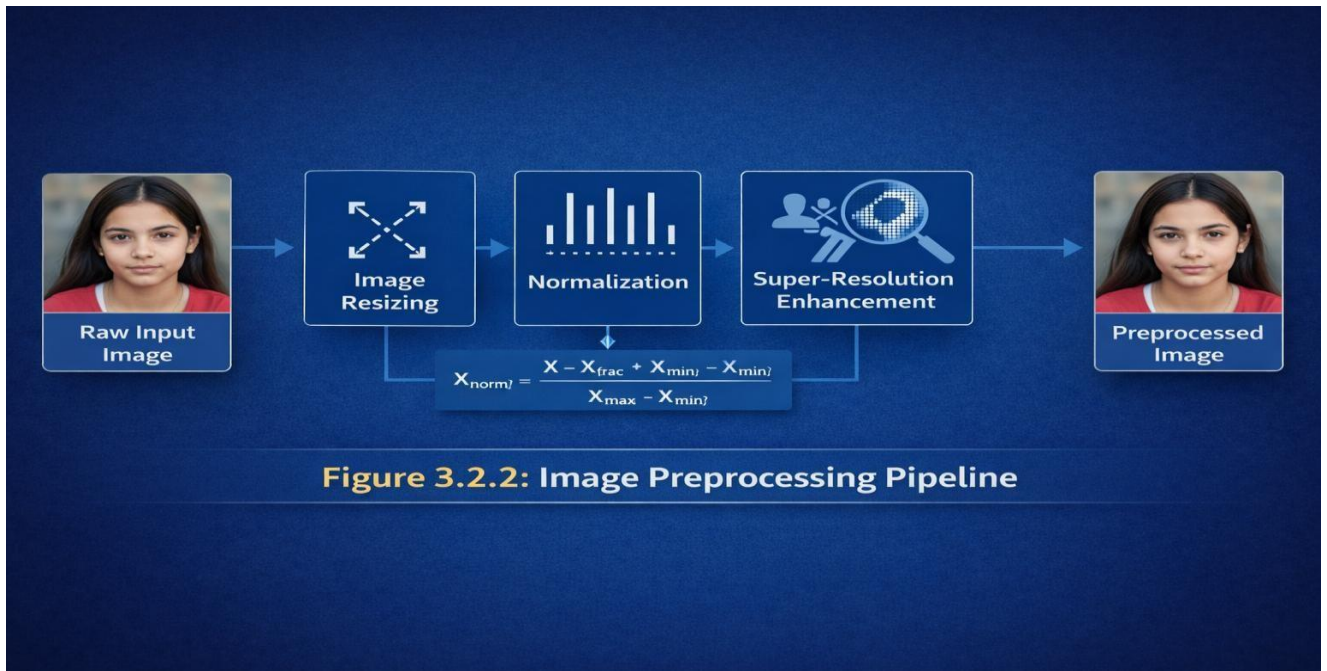


Figure 3.1: Image Preprocessing Pipeline

3.2.3 Face Detection using MTCNN

The system uses the **Multi-task Cascaded Convolutional Neural Network (MTCNN)** for detecting faces within images. MTCNN is widely used because it performs both face detection and facial landmark identification efficiently. It works through a three-stage cascade architecture consisting of the Proposal Network (P-Net), Refine Network (R-Net), and Output Network (ONet). Each stage progressively improves the accuracy of face detection by eliminating incorrect detections.

The P-Net scans the image and identifies candidate face regions. These candidate regions are refined in the R-Net stage by filtering out false positives. Finally, the O-Net produces the final bounding boxes and detects facial landmarks such as eyes, nose, and mouth. This multi-stage detection allows the system to identify faces even under varying lighting conditions, facial expressions, or angles.

MTCNN Three-Stage Cascaded Architecture

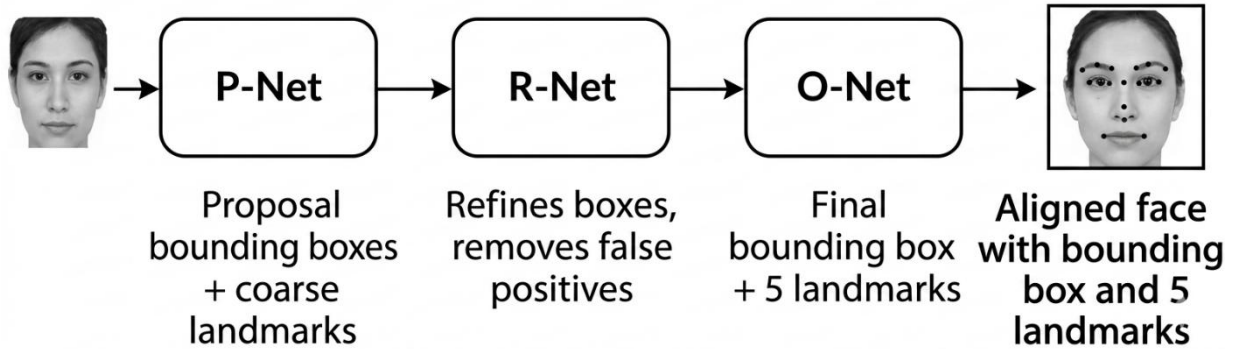


Fig 3.2: *MTCNN Network Stages*

3.2.4 Feature Extraction using Deep Learning

To After detecting faces, the system extracts unique facial features using deep learning models such as FaceNet or VGGFace. These models are convolutional neural networks (CNNs) designed to learn distinguishing patterns from facial images. Instead of storing raw images, the system converts each face into a numerical representation known as a feature vector or embedding.

These embeddings capture important facial characteristics such as distances between facial landmarks and texture patterns. Typically, FaceNet generates a 128-dimensional embedding vector representing each face. These embeddings allow efficient comparison between images and significantly reduce storage requirements compared to storing full images.

Feature Embedding Representation: $f(x) \in \mathbb{R}^{128}$

Model	Model Type	Description
MTCNN	Multitask Cascaded Convolutional Network (MTCNN)	MTCNN extracts deep features from facial images using a cascaded multi-task architecture.

Table 2: Feature Extraction Models

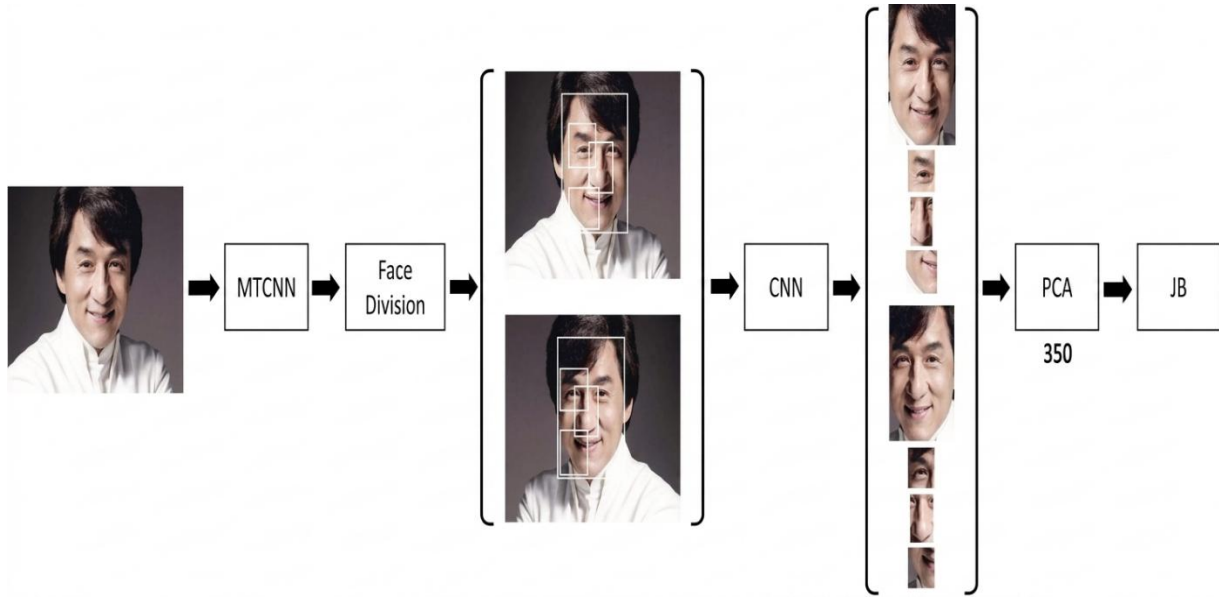


Fig 3.4 Feature Extraction Process

3.2.5 Database Management System

The extracted feature vectors are stored in a centralized NoSQL database such as MongoDB or Elasticsearch. These databases are suitable for handling large volumes of unstructured data and allow efficient storage and retrieval of facial embeddings and metadata. Each record in the database contains the feature vector along with demographic information such as name, age, gender, and last known location.

To ensure fast retrieval, indexing mechanisms are used to organize the stored data efficiently. Security measures such as encryption and role-based access control are also implemented to protect sensitive personal information. The database architecture is scalable, allowing the system to support national-level missing person identification systems.

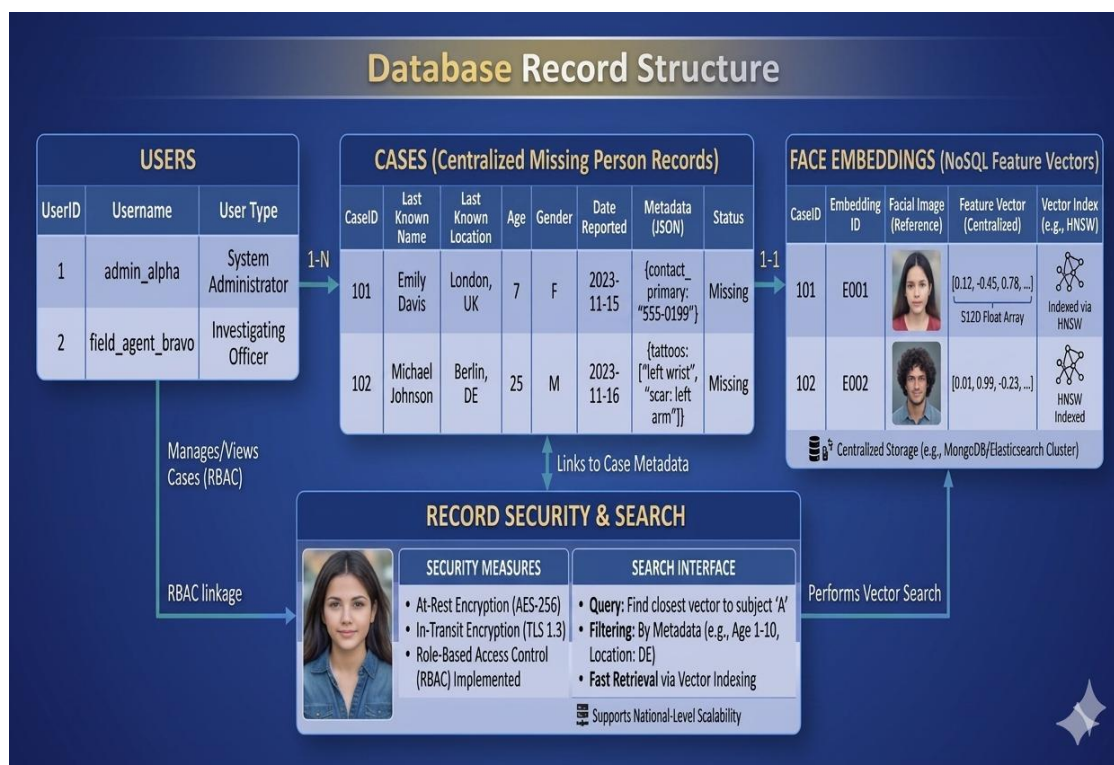


Fig 3.5 Database Record Structure

3.2.6 Matching and Similarity Calculation

When a user uploads a query image, the system extracts its feature vector and compares it with the vectors stored in the database. Similarity metrics such as **Cosine** Similarity or Euclidean Distance are used to measure how closely two facial embeddings match. The system ranks potential matches based on similarity scores, with higher similarity indicating a greater probability of the same person.

Cosine similarity measures the cosine of the angle between two vectors. If the vectors are similar, the cosine value approaches 1. Euclidean distance calculates the direct distance between two vectors in feature space. Smaller distances indicate greater similarity between the facial embeddings.

Cosine Similarity Formula:

$$\text{Similarity} = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

In the proposed missing person identification system, Euclidean Distance is used to measure the similarity between two facial feature vectors. After face detection and feature extraction (using models such as Face-Net or VGG-Face), every face image is converted into a numerical feature vector (for example, a 128-dimensional embedding). These vectors represent the unique characteristics of a person's face. When a new query image is submitted, the system extracts its feature vector and compares it with all stored feature vectors in the database. Euclidean distance calculates the straight-line distance between the two vectors in multidimensional space. If the distance between two vectors is small, the faces are considered similar and likely belong to the same person.

Mathematically, Euclidean distance measures how far apart two feature vectors are. In facial recognition systems, the distance between the query image vector and the stored database vector determines whether they match. A predefined threshold value is used to decide the match. If the computed distance is less than the threshold, the system considers it a potential match; otherwise, it is rejected. This method helps the system rank possible matches and identify the most similar facial records.

Euclidean Distance Formula:

$$d(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

Metric	Calculation Method	Ideal Match Result
Euclidean Distance	Straight-line distance between points.	Low value (close to 0).

Cosine Similarity	Angular judgment between vectors.	High value (close to 1).
--------------------------	-----------------------------------	--------------------------

Table 3: Similarity Metrics

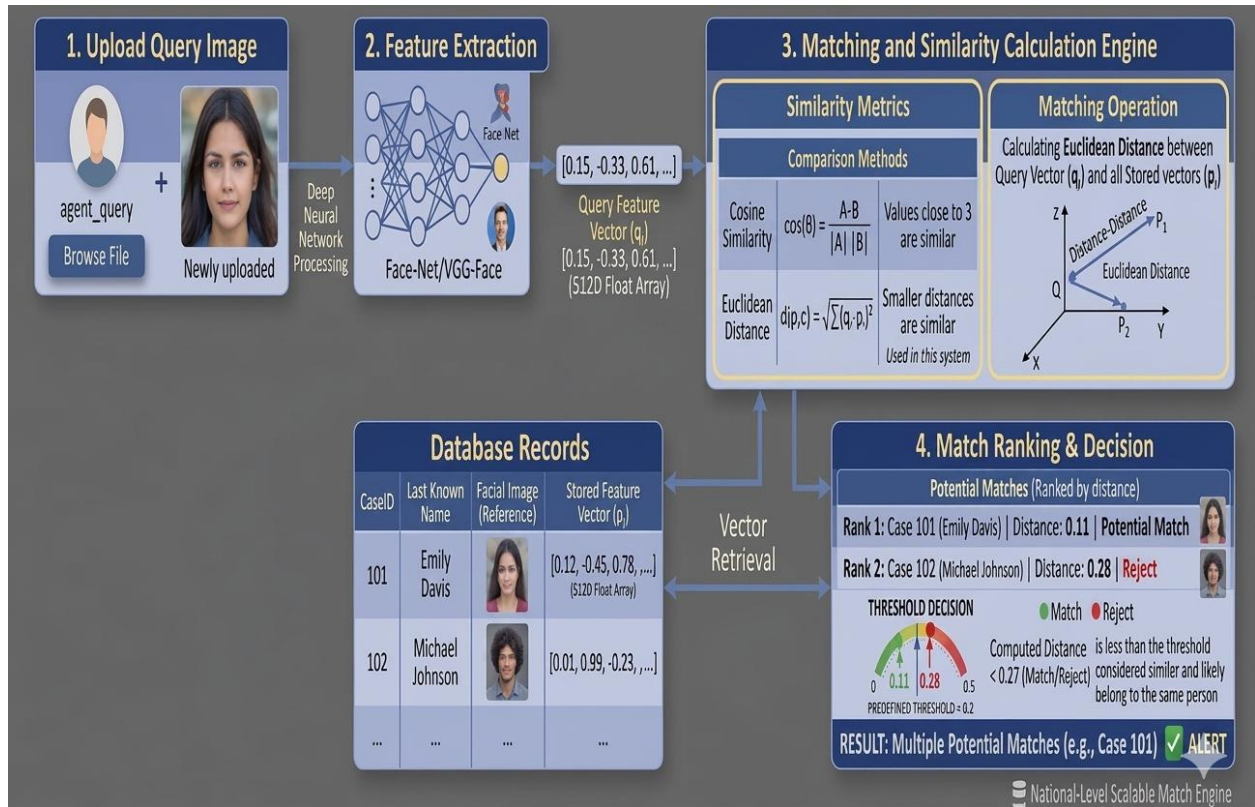


Figure 3.6 Face Matching Process

3.2.7 Verification Process

Dropout After the system identifies potential matches using similarity metrics such as Euclidean distance or cosine similarity, a verification process is performed to confirm the authenticity of the match. The system analyzes additional demographic metadata such as age, gender, and last known location to validate the identified person. This step ensures that the system does not rely solely on facial similarity but also considers contextual information that can improve the reliability of identification.

In cases where the similarity score is close to the threshold or the system is uncertain about the match, manual verification is conducted by authorized personnel such as law enforcement officials

or system administrators. This two-layer verification approach helps reduce false positives and ensures higher accuracy in identifying missing persons. Once verification is completed, the system generates a detailed report containing the matched image, similarity score, demographic information, and last known location of the person.

Parameter	Function
Similarity Score	Initial ranking via Euclidean distance.
Age & Gender	Contextual filters to reduce false positives.
Last Location	Geographic validation for match reliabilit
Manual Review	Human audit for uncertain matches near the threshold.

Table 4: Verification Parameters

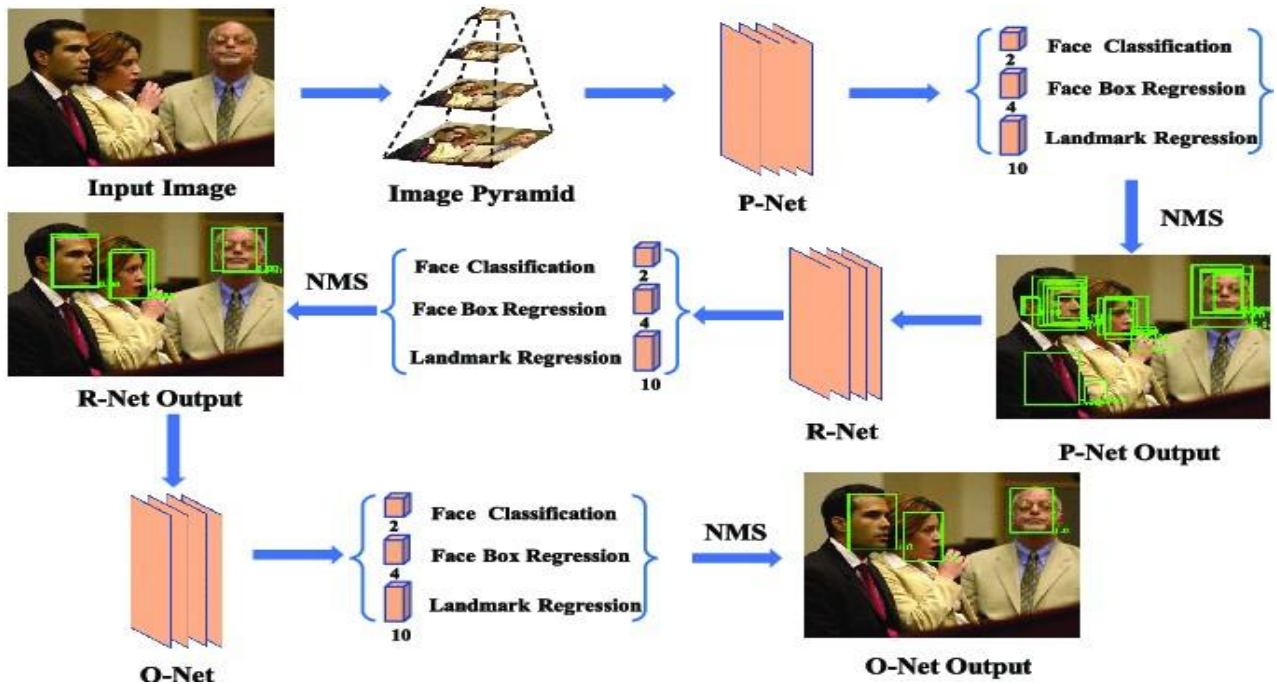


Figure. 3.7 Verification Process

3.2.8 Security Mechanisms

Since the system handles sensitive personal information related to missing individuals, implementing strong security measures is essential. The centralized database stores facial embeddings and personal data, which must be protected from unauthorized access. Encryption techniques are used to secure the stored data and prevent misuse. Access control mechanisms ensure that only authorized users such as law enforcement officials, administrators, or verified NGOs can access the database.

In addition to encryption, the system uses authentication mechanisms such as login credentials and role-based access control (RBAC). Different users are granted different levels of permissions depending on their responsibilities. For example, administrators can manage the database, while general users may only submit reports or search for missing persons. These security mechanisms protect sensitive information and ensure compliance with privacy regulations.

Security Feature	Description
Data Encryption	Protects facial embeddings and personal data from unauthorized access.
Authentication	Users must log in with valid credentials to access the system.
Role-Based Access Control (RBAC)	Different users have different permissions based on their roles.
Secure Database Storage	Sensitive information is safely stored in the centralized database.
Access Monitoring	User activities are tracked to detect unauthorized access.
Data Backup	Regular backups prevent data loss and ensure reliability.

Table 5: Security Features

3.2.9 Additional Features

The proposed system includes several additional features that enhance usability and effectiveness. One important feature is gender and location-based filtering, which allows users to narrow down search results based on specific demographic information. This significantly improves search efficiency when partial information about the missing person is available. Another useful feature is multilingual support, which enables users from different linguistic backgrounds to interact with the system easily.

The system also includes an integrated chat functionality that allows communication between families, law enforcement agencies, and community members. This enables real-time information sharing and collaboration during search operations. Additionally, the system provides automated notifications and messaging services to keep relevant stakeholders updated whenever a potential match is found. These features improve communication, increase public participation, and make the overall missing person identification process more efficient.

3.3 Proposed Architecture

The proposed system follows a multi-stage pipeline that integrates AI and deep learning to efficiently identify missing persons. Data is first collected from multiple sources including police databases, public reports, social media, and crowdsourced submissions, then preprocessed through normalization, resizing, noise removal, and super-resolution to ensure image quality.

The preprocessed images are passed through the MTCNN algorithm for face detection, which identifies facial regions and extracts key landmarks. The detected faces are then processed by deep learning models such as FaceNet or VGG-Face, which convert them into numerical feature vectors that uniquely represent each individual's facial characteristics.

These feature vectors, along with metadata such as name, age, gender, and last known location, are stored in a centralized NoSQL database like MongoDB or Elasticsearch. When a query image is submitted, the system extracts its feature vector and compares it with stored vectors using Euclidean distance or cosine similarity, ranking results by similarity score to identify the most probable matches.

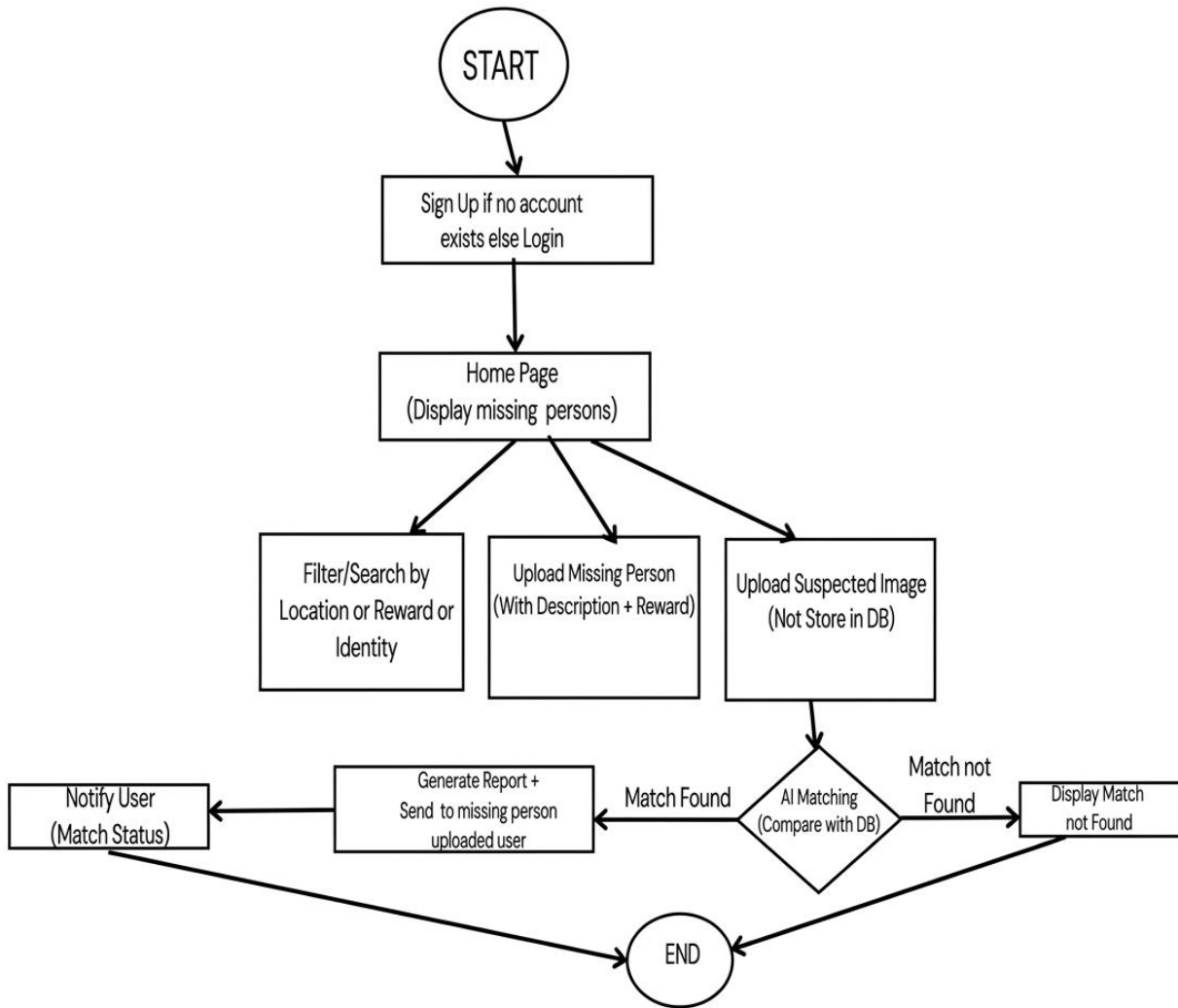


Fig 3.8 An Architecture for Missing Person Identification System

3.3.1. Image Preprocessing

The image preprocessing module enhances the quality of collected images to make them suitable for facial recognition. Images obtained from different sources may vary in resolution, lighting conditions, and orientation. Therefore, preprocessing techniques such as image resizing, normalization, noise removal, and super-resolution enhancement are applied. These steps improve image clarity and ensure consistency across the dataset.

Another important function of this module is the removal of duplicate images and irrelevant data. By eliminating redundant records, the system reduces storage requirements and improves processing efficiency. Proper preprocessing ensures that the images passed to the face detection module are clear and standardized, which significantly improves recognition accuracy.

3.3.2. Face Detection (MTCNN)

The system uses the Multitask Cascaded Convolutional Neural Network (MTCNN) for face detection and landmark localization. It scans images at multiple scales, detects face bounding boxes, and identifies key landmarks such as eyes, nose, and mouth to ensure proper face alignment before feature extraction.

MTCNN operates through three sequential networks — P-Net (candidate region generation), R-Net (false detection removal), and O-Net (final detection and landmark output) — achieving high accuracy with low computational complexity. It performs reliably under challenging real-world conditions including varying angles, lighting, and partial occlusions. The module outputs cropped facial images that are forwarded to the feature extraction stage for recognition.

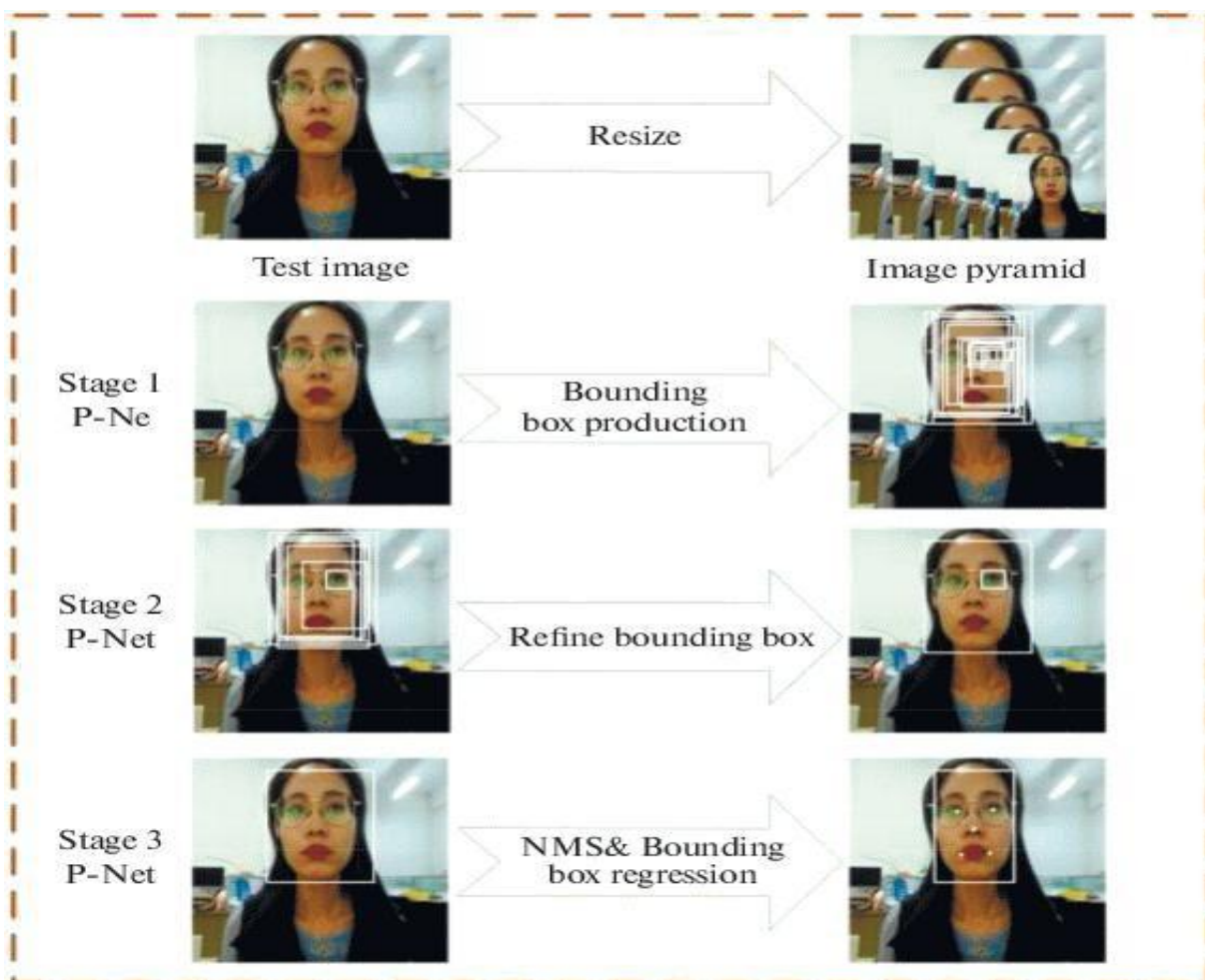


Fig 3.9: MTCNN Face Detection Process

3.3.3. Feature Extraction

After detecting the face, the system extracts unique facial features using deep learning models such as Face-Net or VGG-Face. These models transform the facial image into a numerical representation known as a feature vector or embedding. Each embedding captures the distinctive characteristics of a person's face in a high-dimensional space.

These feature vectors allow the system to efficiently compare faces without storing large image files. Typically, Face-Net produces a 128-dimensional vector that represents each face. This numerical representation simplifies the process of matching and identifying individuals within the database.

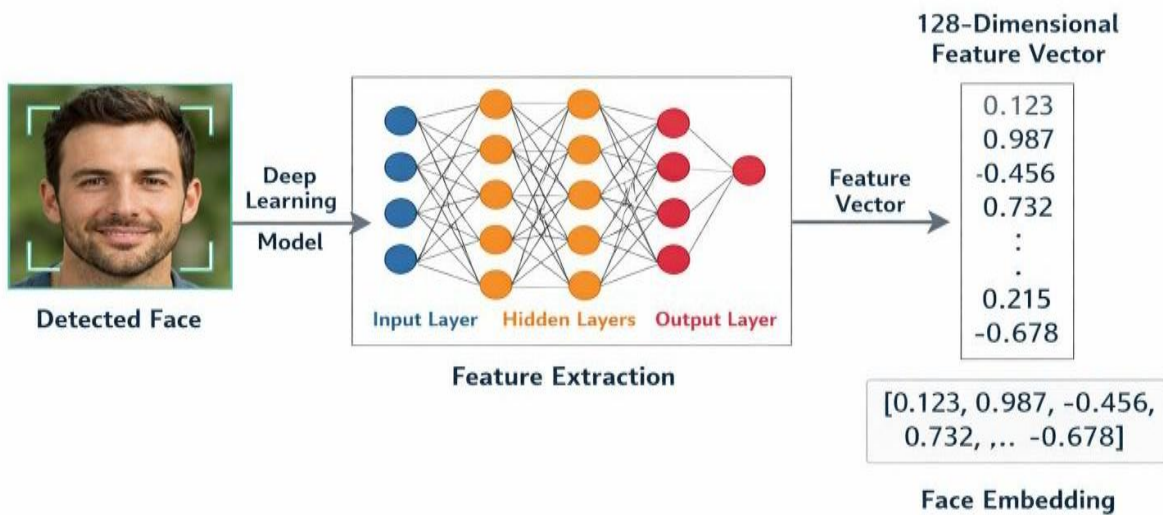


Fig 3.10: Feature Extraction Using Deep Learning Models

3.3.4. Transformer Backbone

The proposed architecture consists of several functional layers, each performing a specific task to enable efficient identification and management of missing person records.

Input Data: The input layer collects facial images and related information such as name, age, gender, and last known location from multiple sources including police records, NGO databases, public reports, and social media. By integrating diverse data sources, the system builds a large dataset that improves the probability of identifying missing individuals.

Image Enhancement: The preprocessing layer standardizes collected images through resizing, normalization, noise reduction, and duplicate removal. Since images from different sources vary in quality, this layer ensures the dataset is clean and consistent, improving the reliability and accuracy of the recognition model.

Face Localization: The face detection layer uses MTCNN to identify facial regions through three sequential stages — P-Net (candidate generation), R-Net (false detection removal), and O-Net (final detection and landmark output). By detecting bounding boxes and key landmarks such as eyes, nose, and mouth, this layer accurately isolates and aligns faces before passing them to the feature extraction stage.

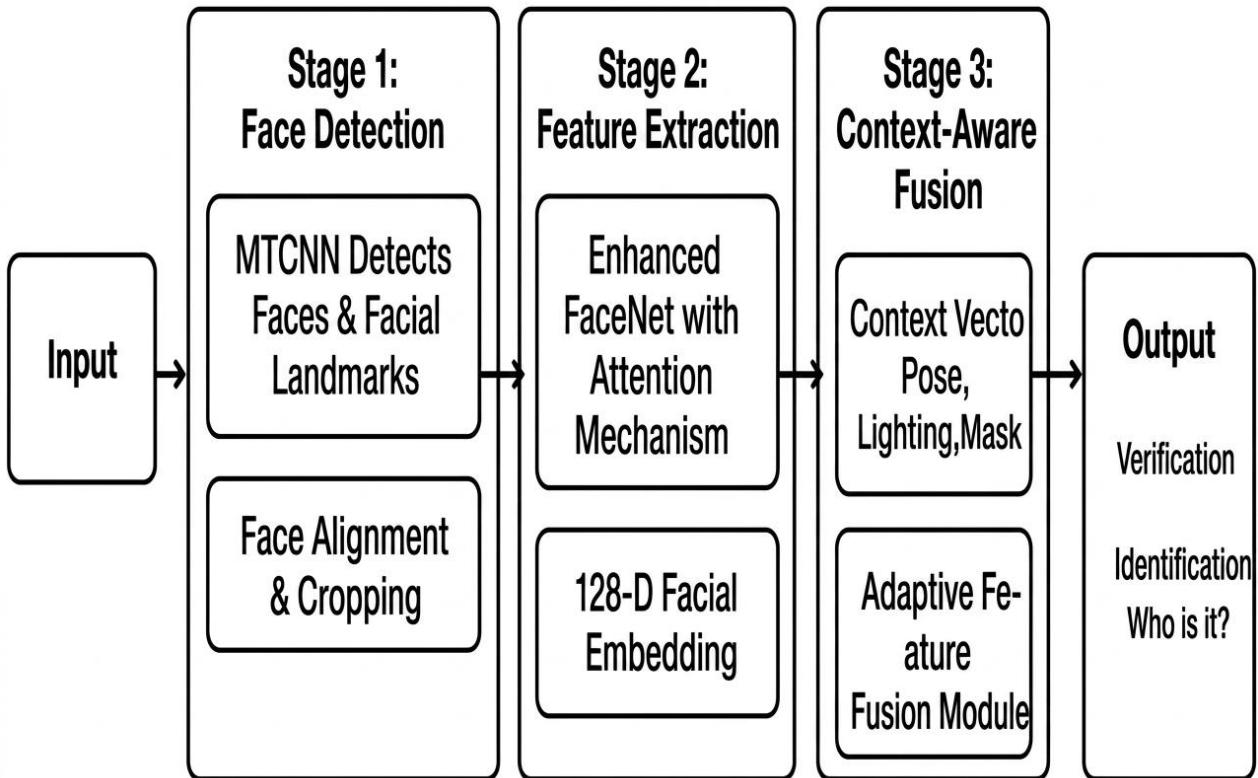


Fig 3.11: Different Stages in MTCNN

Deep Feature Generation:

Following feature extraction layer converts detected faces into numerical representations known as feature vectors or embeddings. Deep learning models such as Face-Net or VGG-Face are used for this purpose. These models analyze facial patterns and produce high-dimensional vectors that uniquely represent each face.

These embeddings capture important facial characteristics and allow efficient comparison between different faces. Instead of comparing full images, the system compares these compact numerical vectors, which significantly reduces computational complexity.

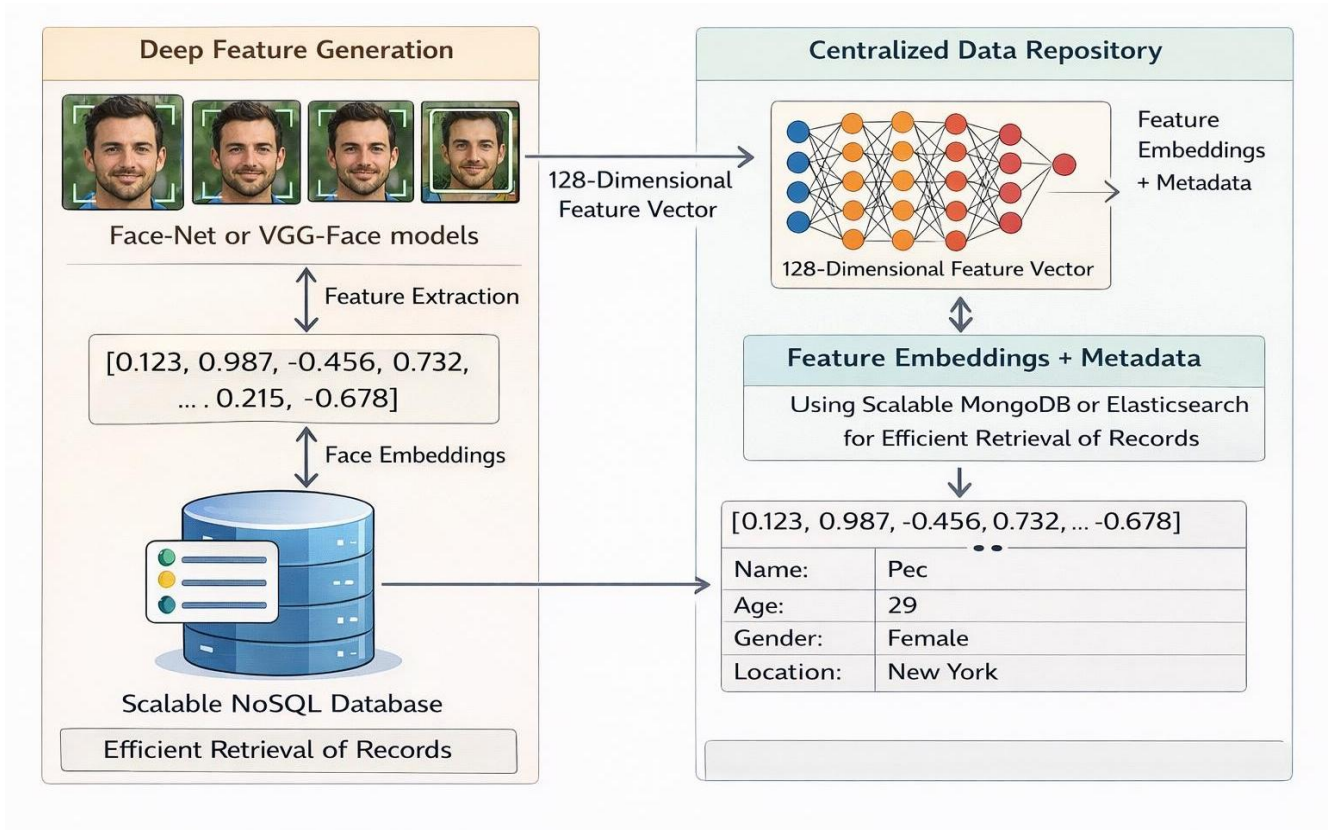


Fig. 3.12: Deep Feature Generation and Centralized Data Repository

Centralized Data Repository:

The storage layer manages the system's centralized database, where facial embeddings and associated metadata are stored. The system uses scalable NoSQL databases such as MongoDB or Elasticsearch to store large volumes of data efficiently.

Each record in the database contains a feature vector along with personal information such as name, age, gender, and location. Indexing techniques are applied to ensure fast retrieval of records during the matching process.

3.4 Features in Proposed System:

3.4.1 Search by Filter

Users can search missing person records using filters like name, age, gender, and location, reducing search time and helping authorities prioritize cases efficiently.

3.4.2 Reward System

Reward information displayed on profiles motivates the public to share useful information, increasing case visibility and improving the chances of locating missing persons quickly.

3.4.3 Messaging System

The messaging system enables secure real-time communication between families, authorities, NGOs, and volunteers, allowing users to share sighting details, location information, and coordinate search activities effectively.

3.4.4 Multi-Language Translator

This feature automatically translates reports and messages into multiple regional languages such as Hindi, Telugu, and Tamil, making the system accessible to users from diverse linguistic backgrounds and broadening the reach of search efforts.

3.4.5 Image/Face Recognition

AI-based facial recognition detects faces, extracts unique facial features as numerical feature vectors, and compares them against the database to accurately and quickly identify missing persons, outperforming traditional manual methods.

3.4.6 NGO Support

NGOs can register on the platform to upload cases, share updates, and coordinate with families and authorities. Their volunteer networks help distribute information widely, making the system more collaborative and effective in locating missing individuals.

3.5 System Requirements

Hardware Requirements:

- A GPU with atleast 8GB memory.
- A system with atleast 8GB RAM.

Software Requirements:

- Python – 3.10.2
- Django – 4.2
- Node.js – 20
- Flask – 2.3
- Express.js – 4.18
- Pytorch - 2.1
- MongoDB – 6.0

Python

Python is a high-level, interpreted programming language known for its simplicity, readability, and vast ecosystem of libraries. It supports multiple programming paradigms including procedural, object-oriented, and functional programming, making it widely used in software development, data analysis, and machine learning.

Django

Django is a high-level, open-source web framework written in Python that enables rapid development of secure and scalable web applications. It follows the Model–View–Template (MVT) architecture and provides built-in features such as user authentication, URL routing, and database management, reducing development effort significantly.

Flask

Flask is a lightweight and flexible Python web framework for building web applications and APIs. Its minimalistic design and extensive extension ecosystem allow developers to create applications quickly and efficiently, making it suitable for projects of all sizes.

PyTorch

PyTorch is an open-source deep learning framework known for its dynamic computational graph, Pythonic interface, and automatic differentiation feature. It supports GPU acceleration and distributed training, enabling efficient development and deployment of complex neural network models for both research and production.

MongoDB

MongoDB is a NoSQL database designed for storing and managing large volumes of unstructured or semi-structured data. Its flexible document-based model, scalability, and high performance make it well suited for applications requiring fast data retrieval and efficient storage of diverse data types.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system architecture represents the overall structure of the proposed missing person identification system. It describes how different components interact with each other to perform the identification process using facial recognition technology.

The architecture mainly consists of users, an image processing module, a facial recognition engine, and a centralized database. The system allows family members, friends, or public users to upload images of missing persons. These images are stored in the centralized database along with personal information such as name, age, gender, and location.

When a public user captures or uploads an image of a suspected missing person, the system extracts facial features from the image using deep learning techniques. The extracted features are then compared with the facial feature vectors stored in the database.

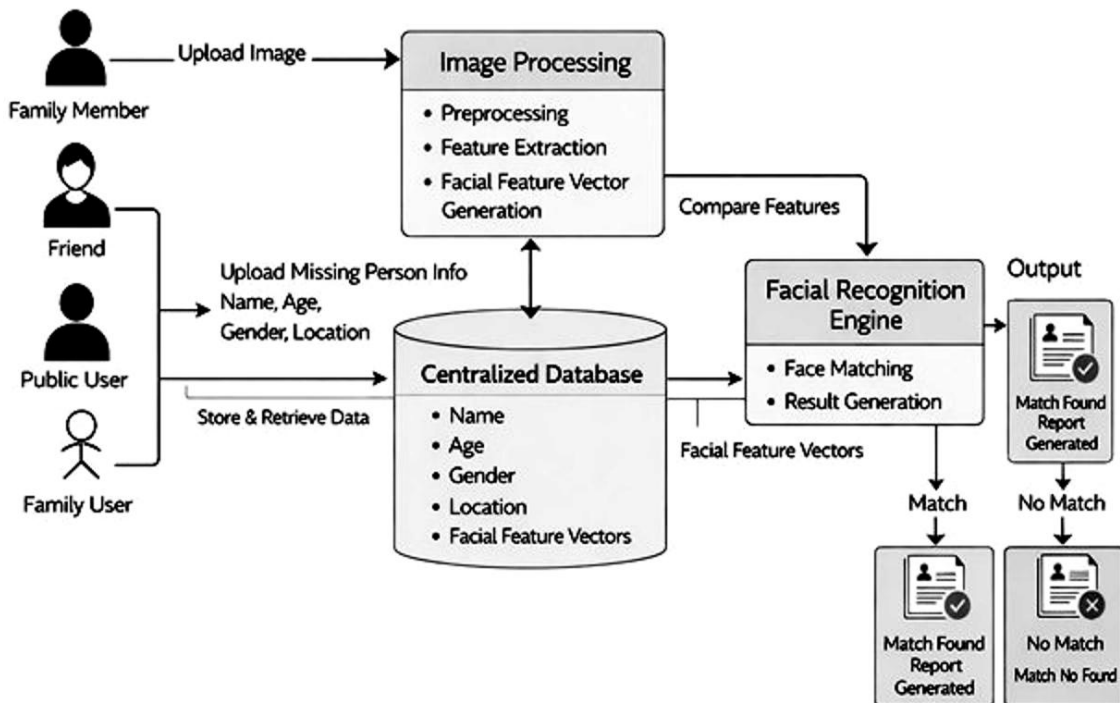


Fig 4.1: System Architecture

If the system finds a match, it generates a report and notifies the relevant authorities and family members. If no match is found, the system records the information for future analysis. This architecture ensures efficient identification and tracking of missing persons through automated facial recognition.

4.2 Flow Chart Diagram

The flowchart diagram represents the sequence of operations performed by the missing person identification system. It explains the step-by-step process from user registration to the identification result.

The process begins when a user opens the system and creates an account. If the user already has an account, they can directly log in to access the homepage. The homepage displays information about missing persons currently registered in the system.

The user can perform different operations such as searching missing persons by location or identity, uploading information about a missing person, or uploading a suspected person's image for identification.

When an image is uploaded, the artificial intelligence model analyzes the image and compares it with the images stored in the database. If a matching face is found, the system generates a report and sends it to the person who uploaded the missing person information.

If no match is found, the system displays a "Match Not Found" message and stores the uploaded image for future comparison. Finally, the process ends after notifying the user about the result.

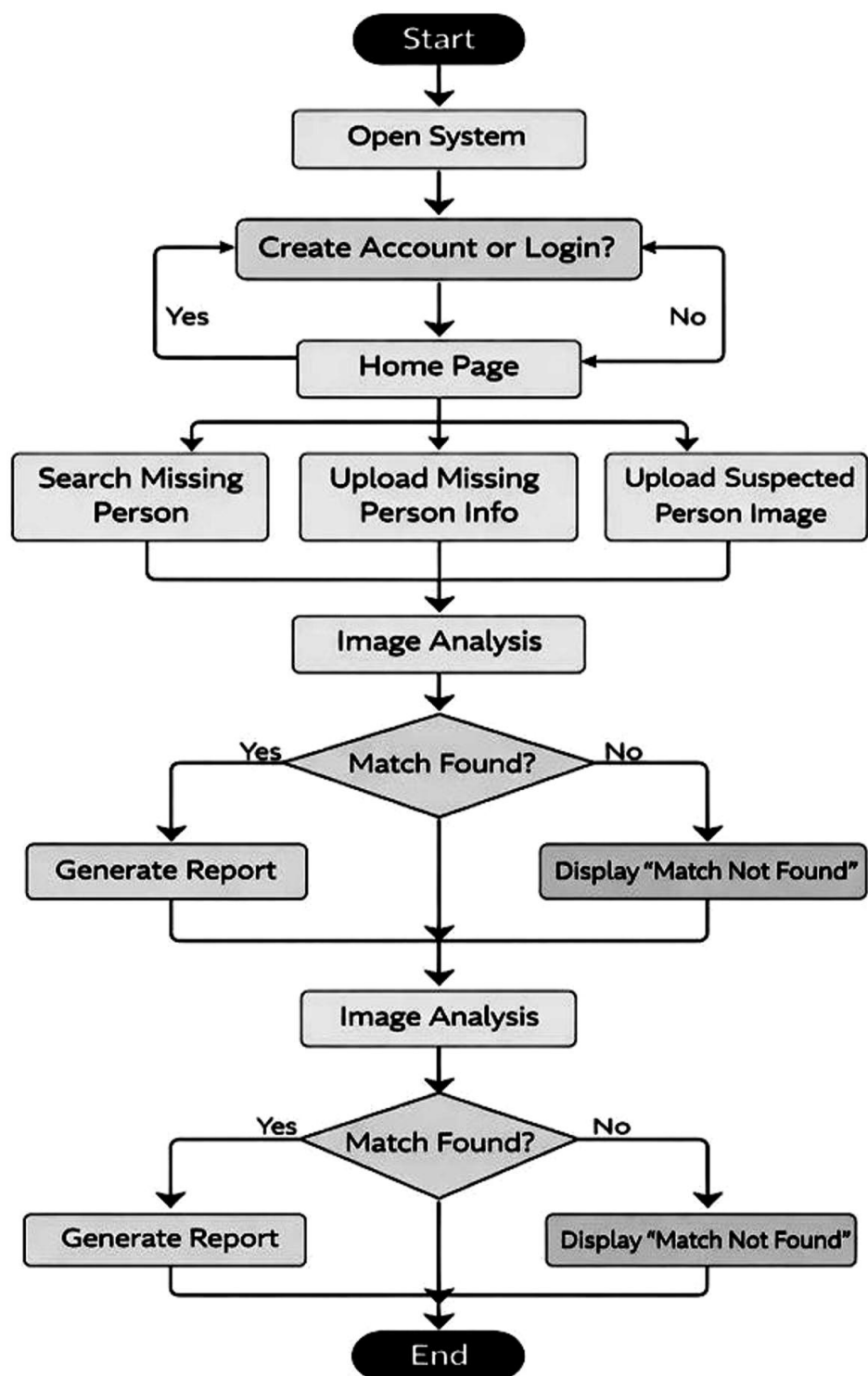


Fig 4.2: Flow Chart

4.3 UML Diagrams

Unified Modeling Language (UML) diagrams are used to visually represent the design and functionality of the system. These diagrams help in understanding how the system operates and how different components interact with each other.

4.3.1 Use Case Diagram

The use case diagram illustrates the interaction between users and the system. It shows the different actions that users can perform within the system.

In the proposed system, there are mainly three types of users: family members, public users, and administrators. Family members can upload images and details of missing persons. Public users can upload images of suspected individuals they encounter. The system processes these images using facial recognition algorithms to determine whether there is a match with the existing database.

If a match is found, the system generates a report and notifies the concerned authorities and family members. The administrator manages the database, verifies records, and monitors the system.

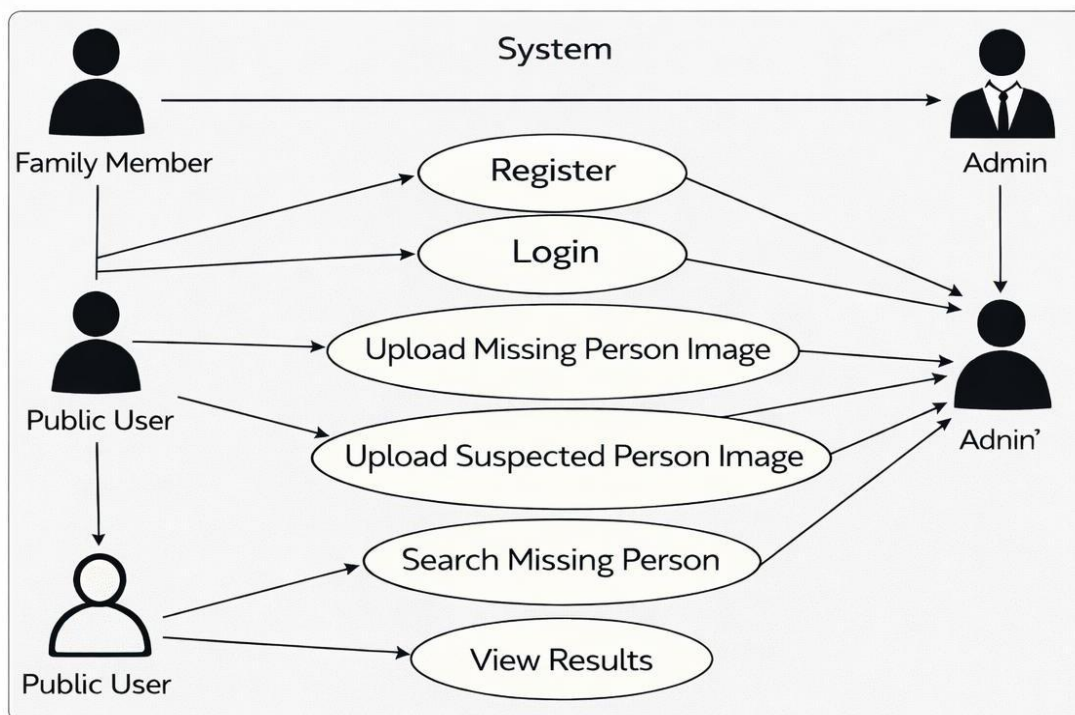


Fig 4.3: Use Case Diagram

4.3.2 Class Diagram

The class diagram represents the structure of the system by showing different classes, their attributes, and the relationships between them.

In this system, important classes include User, MissingPerson, ImageData, FaceRecognitionModel, Database, and Report. The User class contains information about registered users such as user ID, name, and login details. The MissingPerson class stores information about missing individuals including personal details and images.

The FaceRecognitionModel class processes images and extracts facial features for comparison. The Database class stores all records related to missing persons and facial feature vectors. The Report class generates identification reports when a match is found.

The relationships between these classes help in organizing the system structure and maintaining efficient data management.

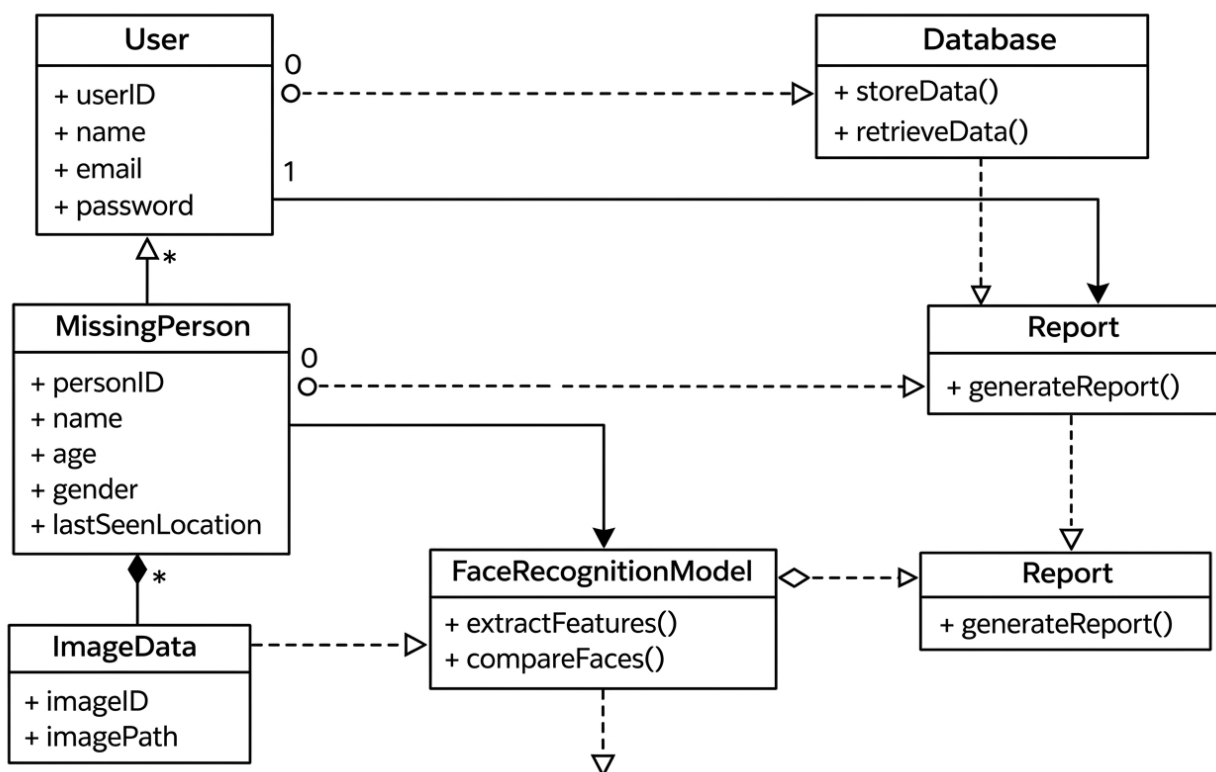


Fig 4.4: Class Diagram

4.3.3 Sequence Diagram

The sequence diagram describes the interaction between different system components in a sequential order.

The process begins when the user uploads an image. The system sends the image to the facial recognition module, which extracts facial features using deep learning models. These features are then compared with the stored feature vectors in the database.

If the similarity score exceeds a predefined threshold, the system identifies the person as a match. The system then generates a report and sends notifications to relevant users. If no match is found, the system stores the image for future comparisons.

This diagram helps in understanding the step-by-step communication between the user interface, recognition model, and database.

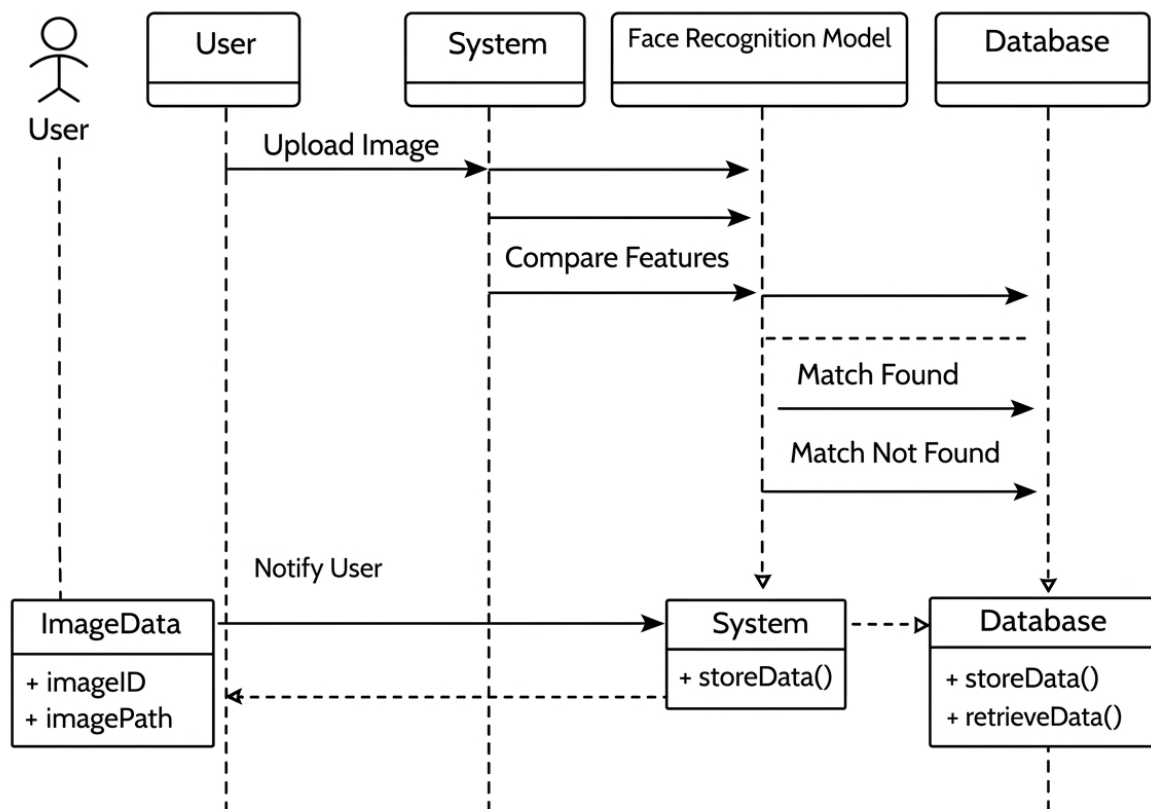


Fig 4.5: Sequence Diagram

4.3.4 Activity Diagram

The activity diagram represents the workflow of the system and the sequence of activities involved in the identification process.

The process begins with user registration or login. After logging in, the user can upload images or search for missing persons. When an image is uploaded, the system processes the image, detects the face, extracts facial features, and compares them with the database.

If a match is found, the system generates a report and notifies the user. If no match is found, the system displays a message indicating that no match was detected. The activity diagram clearly illustrates the decision points and flow of actions in the system.

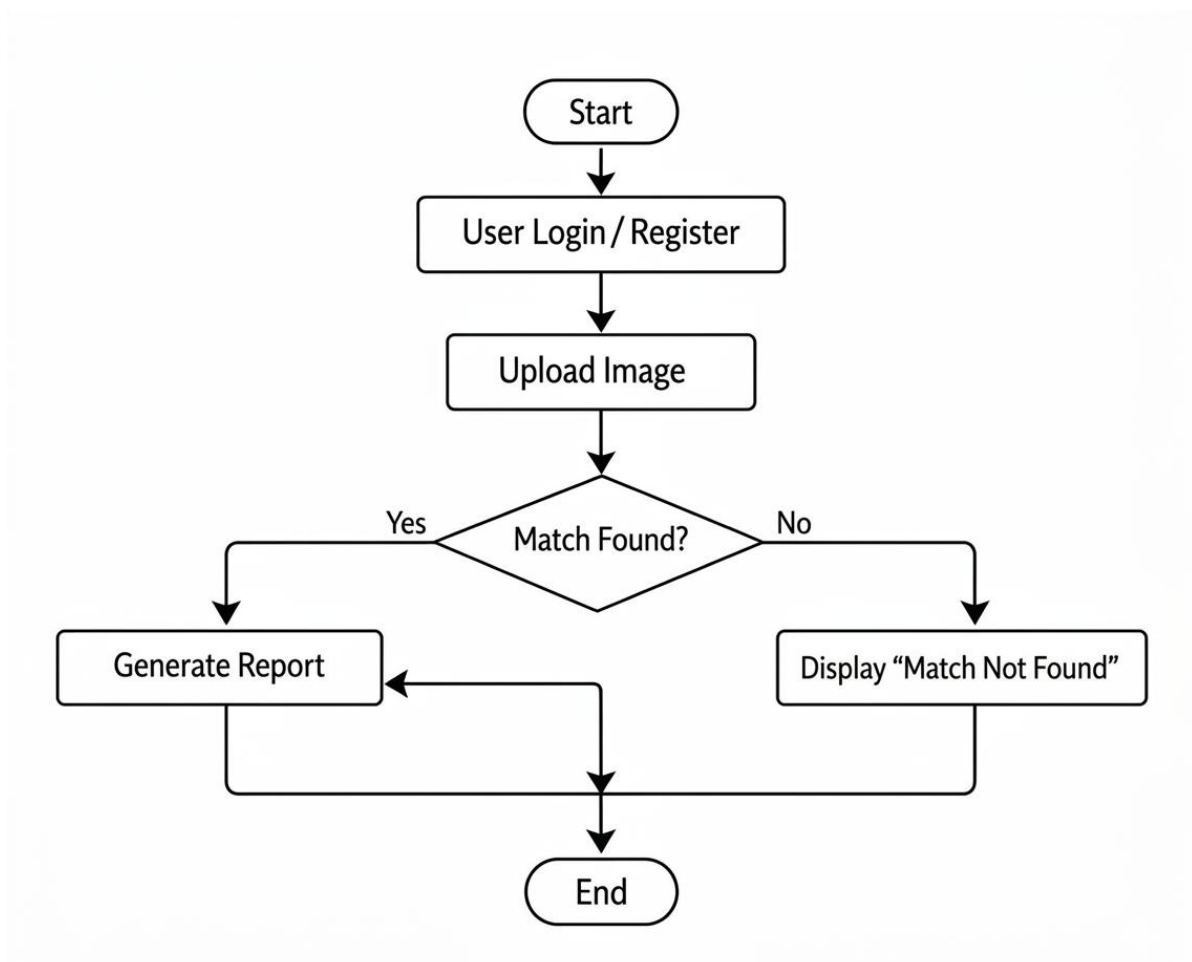


Fig 4.6: Activity Diagram

CHAPTER 5

SYSTEM TESTING

5.1 Purpose of Testing

The purpose of testing is to ensure that the missing-person identification system works correctly, reliably, and securely before real-world use. It helps detect and fix defects in modules like image upload, face detection, matching, chat, and case management.

Testing also verifies performance, security, and role-based access control, ensuring smooth operation and accurate results. It reduces system failures and increases user confidence.

5.2 Types of Testing

5.2.1 Unit Testing

Unit testing verifies individual modules such as registration, login, image upload, face detection, database operations, and notifications. Each module is tested with valid and invalid inputs to ensure correct functionality, error handling, and reliability.

It helps in early detection of bugs at the development stage, making them easier and cheaper to fix. This testing also ensures that each component works independently as expected.

5.2.2 Integration Testing

Integration testing checks interaction between modules, especially the flow from image upload to face matching. It ensures smooth data transfer between frontend, backend, and database without errors or inconsistencies.

It also validates that combined modules function correctly as a group. This reduces issues caused by incorrect data formats or communication failures between components.

5.2.3 User Interface Testing

UI testing evaluates the design and usability of pages like login, upload, and chat screens. It ensures proper layout, navigation, responsiveness, and clear user messages for better user experience.

It also checks accessibility and consistency across different devices and browsers. This helps users interact with the system easily without confusion.

5.2.4 System Testing

System testing validates the complete application by executing real-world workflows like case registration, matching, and reporting. It ensures all features work together correctly under normal conditions.

It also checks system behavior under different scenarios and verifies that all requirements are fully satisfied. This ensures the system is ready for deployment.

5.2.5 Performance Testing

Performance testing measures system speed, scalability, and stability under heavy load. It evaluates response time, concurrent user handling, and identifies performance bottlenecks.

It also ensures that the system performs efficiently as the number of users and data increases. This helps maintain a smooth and fast user experience.

5.2.6 Security Testing

Security testing ensures data protection and proper access control. It checks authentication, prevents unauthorized access, and tests vulnerabilities like SQL injection and XSS.

It also verifies data encryption and secure communication between users and the system. This protects sensitive information from misuse or attacks.

White Box Testing

White box testing examines the internal structure and logic of the system to test code-level functionality.

It ensures that all paths, conditions, and loops in the code are properly tested. This helps in improving code quality and detecting hidden errors.

Black Box Testing

Black box testing evaluates the system based on inputs and outputs without knowledge of internal implementation.

It focuses on validating functionality against requirements. This ensures that the system behaves correctly from the user's perspective.

CHAPTER 6

RESULTS & DISCUSSIONS

6.1 Accuracy and Loss

In this project, the performance of the MTCNN based facial recognition model was evaluated by analysing over multiple training epochs. During training, the accuracy gradually increased from a low initial value to around **90%** on the training set and **87%** on the validation set, while the corresponding loss decreased steadily from a high initial value to approximately **0.25** for training and **0.30** for validation. The accuracy and loss curves showed complementary behaviour: as the model learned more discriminative facial features, accuracy improved and loss reduced, then both metrics stabilised after a certain number of epochs, indicating convergence. The absence of a large gap between training and validation accuracy, and similarly between their loss values, suggests that the model generalised well to unseen data without severe overfitting, which is essential for reliable missing person identification in real world conditions.

This Beyond model-level metrics, system-level statistics showed that integrating this AI framework improved real-world performance of missing-person case handling. After introducing the MTCNN-based identification and reward-driven community participation, the resolution rate increased each year compared to the non-AI baseline, demonstrating practical benefits of higher recognition accuracy and faster processing. These gains, combined with gender/location filtering, multilingual support, automatic messaging, and NGO-backed rewards, confirm that the system's technical accuracy translates into better outcomes in actual search and reunification effort

6.2 PREDICTED CAPTIONS

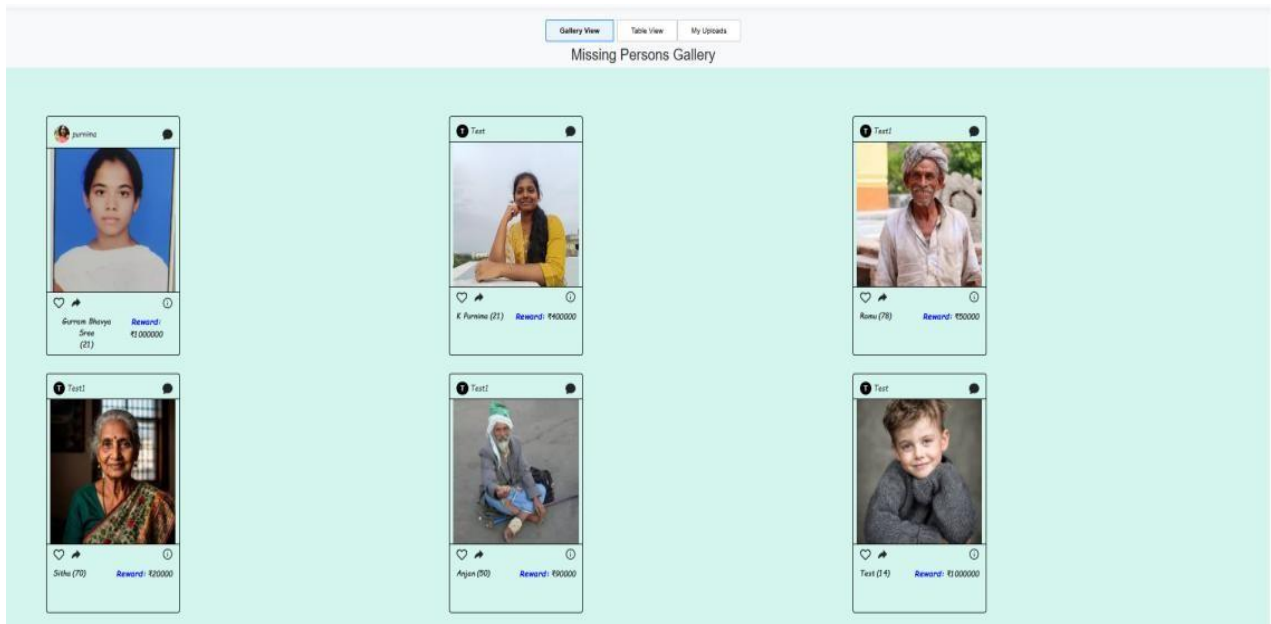


Fig6.1: Home Page with Missing Persons

The image shows a web application interface for uploading missing person information. At the top, there is a navigation bar with links: Home, About, Upload, Matches, NGO Support, and Faqs. There is also a 'Select Language' dropdown menu. Below the navigation bar, the title 'Upload Missing Person Information' is centered. Below the title, there is a sub-header 'Help us reunite families by submitting details and a photo of a missing person.' and a note 'Your submission will be securely stored and matched using AI-powered face recognition. Please provide clear images and accurate information to improve search results.' The main form is titled 'Upload Missing Person Info' and contains the following fields: Name* (text input), Age* (text input), Gender* (dropdown menu with 'Male' selected), LandMark* (text input), City* (text input), State* (dropdown menu), Pincode* (text input), Reward* (text input), Photo* (file upload button), and Description (Optional) (text input). At the bottom of the form is an 'Upload' button.

Fig 6.2: Upload Missing Person Details

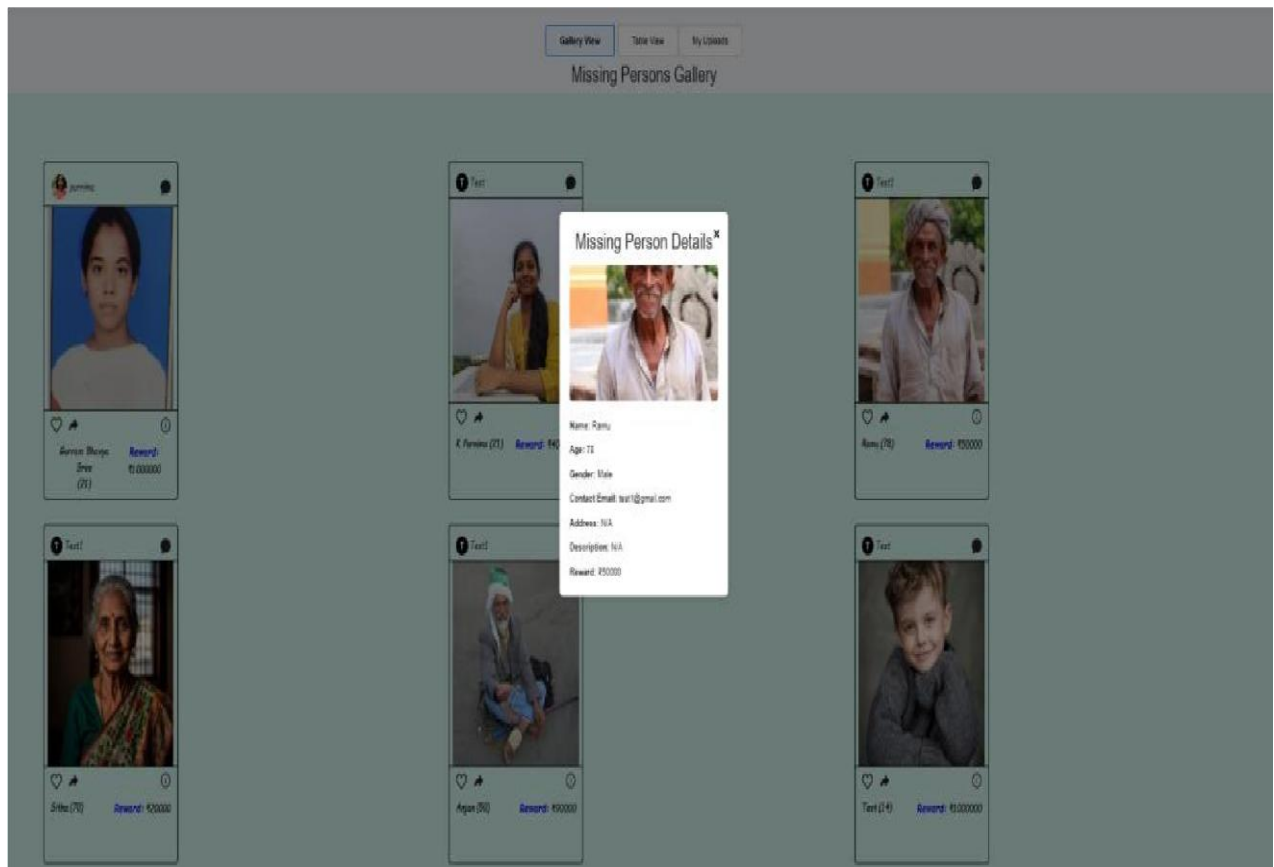


Fig 6.3: Missing Person Information

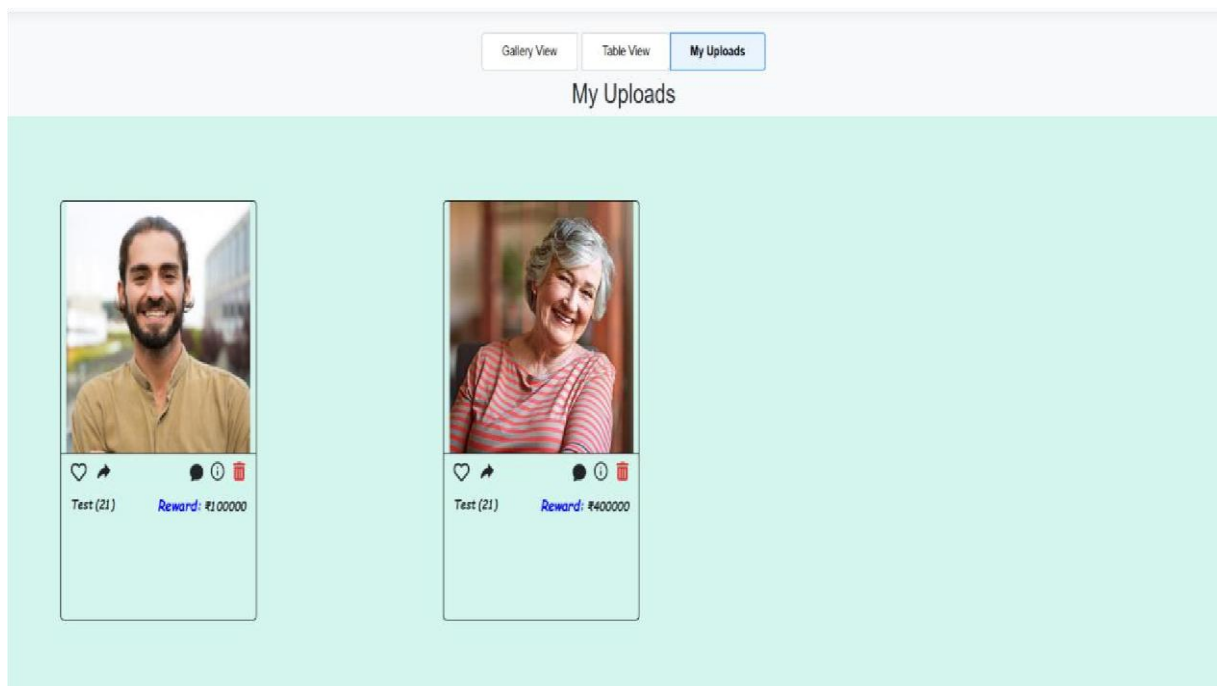


Fig 6.4: My Uploads

Gallery View

Table View

My Uploads

Missing Persons Gallery

Search...

All Genders

All States

Photo	Name	Age	Gender	Reward	Description	Address	Likes
	Gurnam Bhanya Sree	21	Female	₹100000	—	Guntur , Andhra Pradesh	7
	K. Punima	21	Female	₹40000	Last Seen Location: Coka Beach	Paraji , Goa	0
	Ramu	78	Male	₹50000	—	Hyderabad , Telangana	0
	Silma	70	Female	₹20000	—	LB Nagar , Telangana	0
	Arjan	50	Male	₹50000	—	Banglore , Karnataka	0
	Test	14	Male	₹100000	—	Noida , Uttar Pradesh	0
	Bhanya Sree	57	Female	₹40000	—	Guntur , Andhra Pradesh	0

Rows per page: 10

1-7 of 7

<< < > >>

Fig 6.5: Search By Filter

హెల్ప్ షేజ్ మా గురించి అప్‌లోడ్ చేయండి మ్యాచులు N60 మధ్య తరచుగా అడిగే ప్రశ్నలు

Telugu

AI మిస్సింగ్ వర్షన్ ఫైండర్ అంటే ఏమిటి?

AI మిస్సింగ్ వర్షన్ ఫైండర్ అనేది AI-ఆధారిత ఇమేజ్ మ్యాచింగ్ మరియు కంప్యూటర్ సహజాన్ని ఉపయోగించి తప్పిపోయిన వ్యక్తులను వారి కుటుంబాలతో తిరిగి కలవడానికి సహాయపడే ఒక వేదిక.

ఈ వ్యవస్థ తప్పిపోయిన వ్యక్తులను ఎలా గుర్తిస్తుంది?

చిత్రాలను ఎవరు అప్‌లోడ్ చేయవచ్చు?

నా డేటా సురక్షితంగా ఉందా?

AI మ్యాచింగ్ ఎంత ఖచ్చితమైనది?

తప్పిపోయిన వ్యక్తుల గురించి నవీకరణలు, ముఖ్యమైన వార్తలు మరియు హెచ్చరికలను పొందడానికి నట్రి నైట్ చేయండి.

మీ ఇమేయిల్

సభ్యత్వం సొందండి

© 2026 AI తప్పిపోయిన వ్యక్తిని కనుగొనే సాధనం. అన్ని హక్కులూ రక్షితం.

Fig 6.6: Multi Language Translator

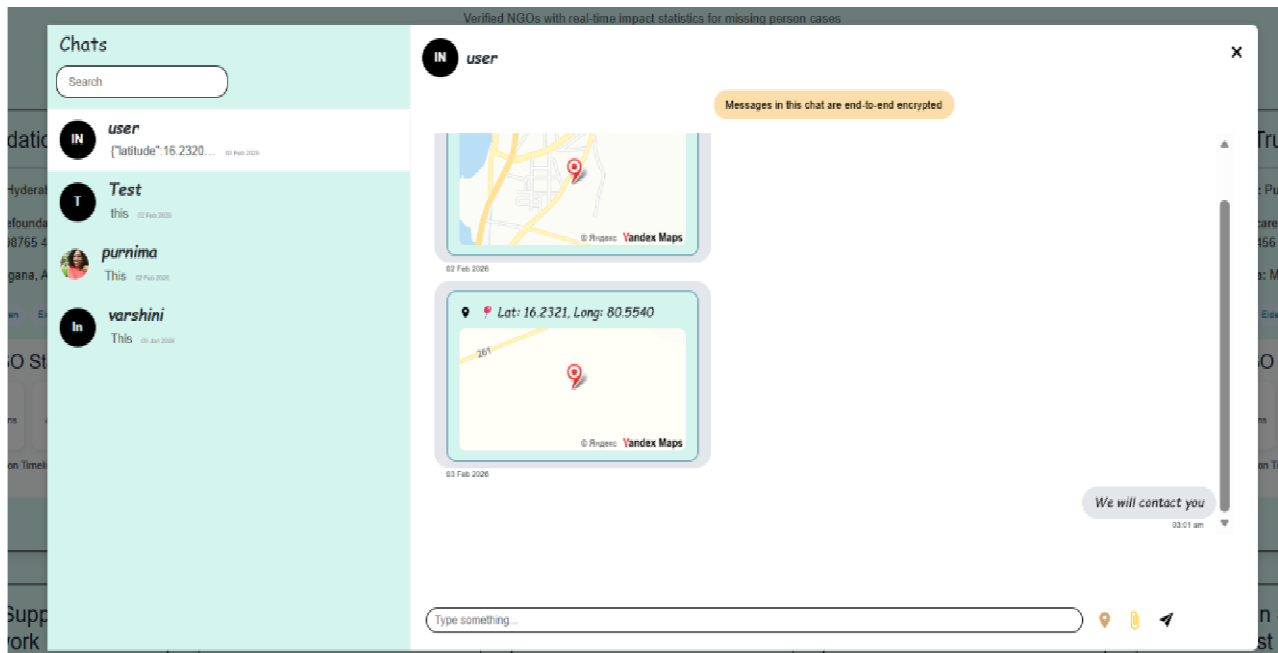


Fig 6.8: Messenger (Text, Location ,Documents/Pictures Sharing)

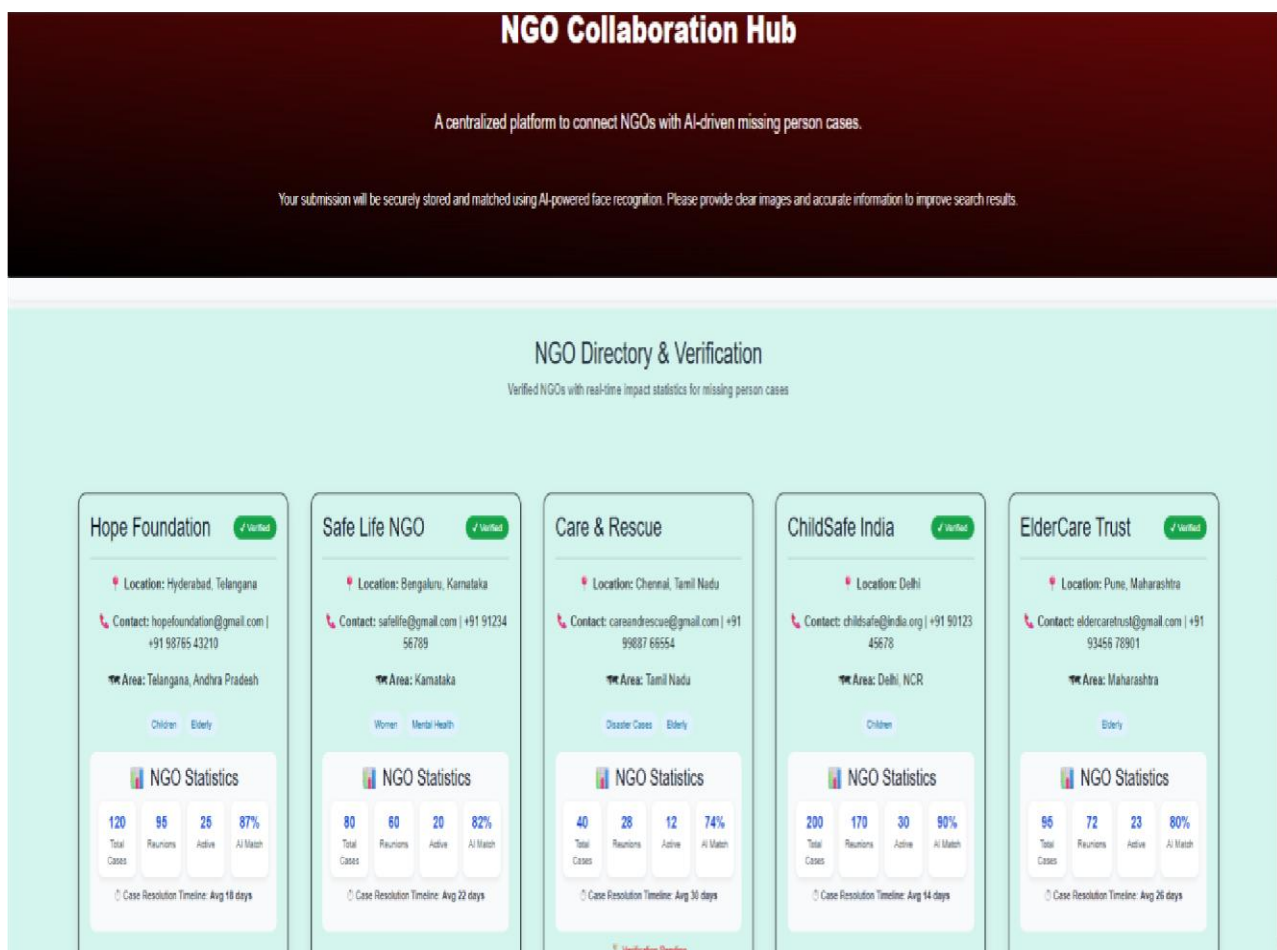


Fig 6.7: NGO Support

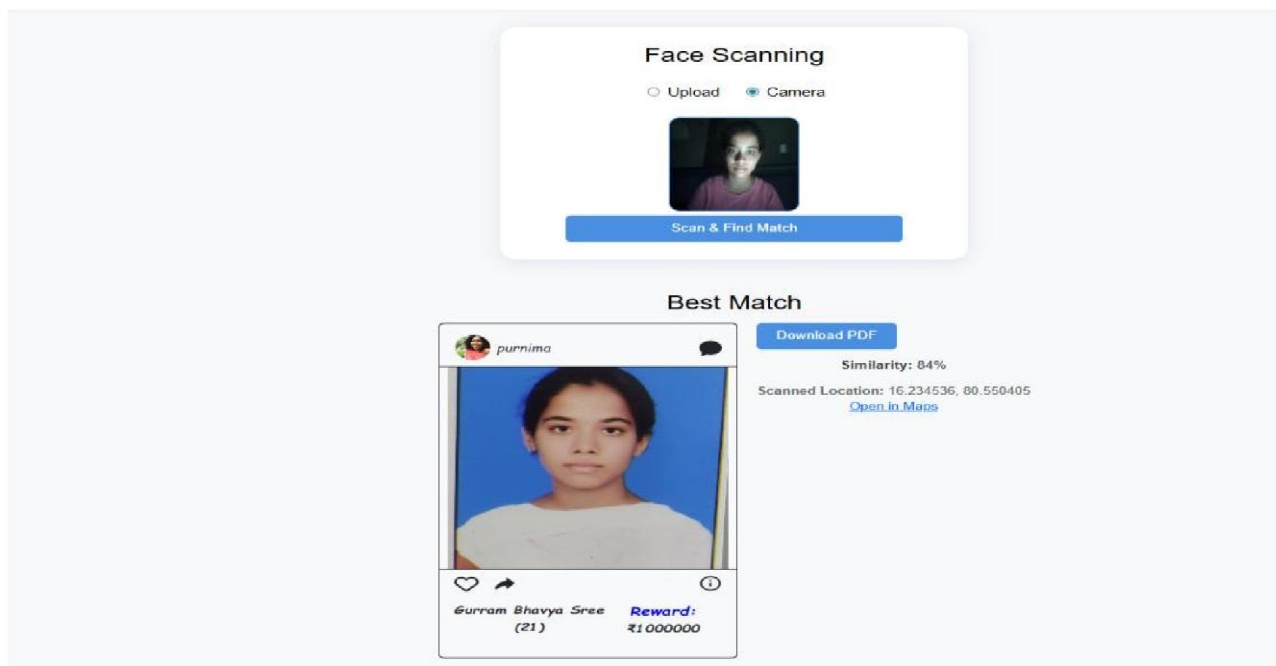


Fig 6.9: Match Finding

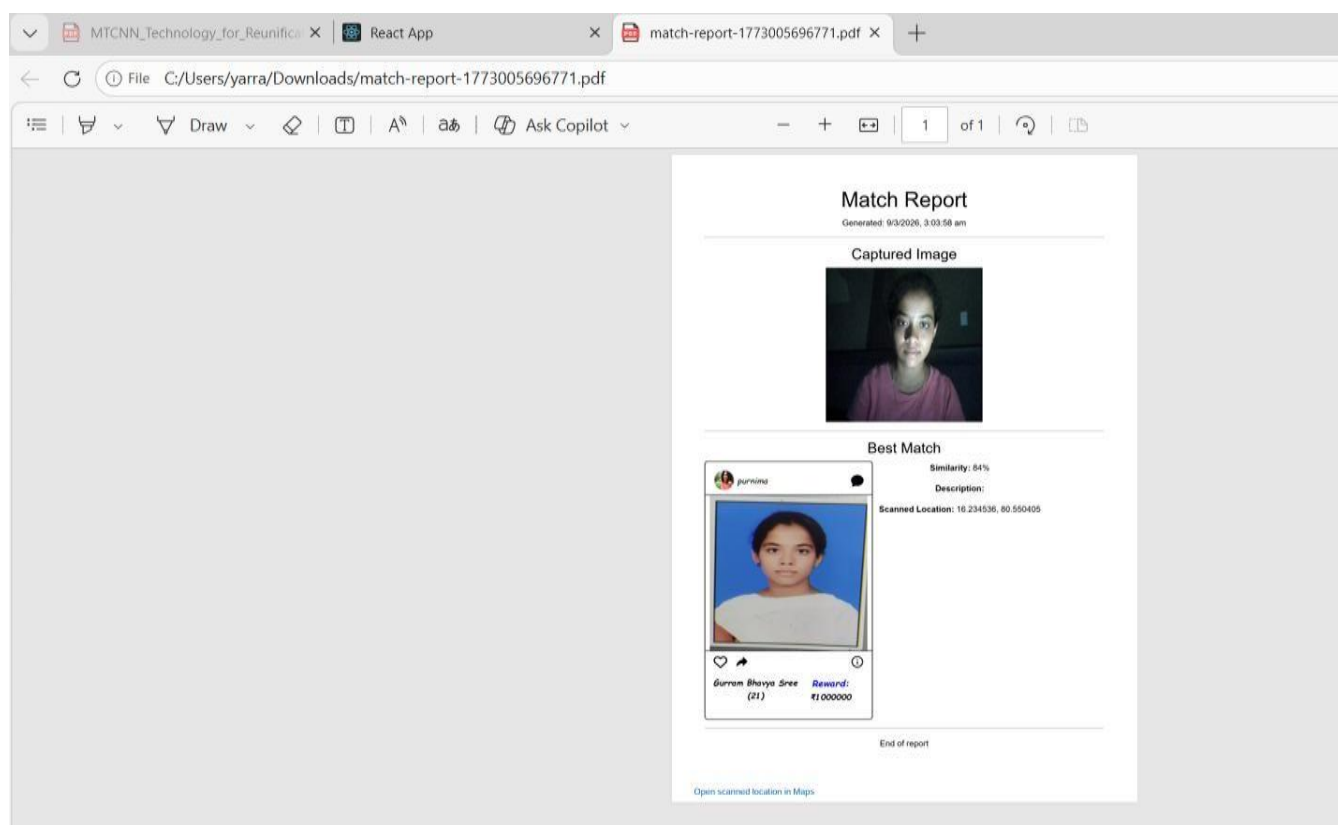


Fig6.10: Generated Report

CHAPTER 7

DEPLOYMENT

7.1 Introduction

Deployment is the process of making the developed system available for users to access and use in a real-world environment. After the successful development and testing of the missing person identification system, the next step is to deploy the application so that it can be used by family members, authorities, and public users to identify missing persons.

The deployment phase ensures that the system is properly installed, configured, and integrated with the required hardware and software components. This stage also verifies that the system performs efficiently when accessed by multiple users and processes real-time image data.

Additionally, deployment ensures that the system is stable and ready for continuous operation. It bridges the gap between development and real-world usage.

7.2 Deployment Environment

The missing person identification system is deployed in a web-based environment so that users can access the system through any device with an internet connection. The deployment environment includes both hardware and software components required to run the system smoothly.

It ensures that all system components work together efficiently under real-time conditions. Proper environment setup helps avoid compatibility and performance issues.

Hardware Requirements

The system requires the following hardware components:

- A computer or server with sufficient processing power
- Minimum 8 GB RAM for smooth image processing
- High-speed internet connection
- Storage space for image datasets and database

These hardware components ensure faster processing of images and better system performance. Adequate resources help handle large datasets effectively.

Software Requirements

The system uses the following software technologies for deployment:

- Operating System: Windows / Linux
- Programming Language: Python
- Web Framework: Flask or Django
- Database: MySQL / SQLite
- Libraries: OpenCV, NumPy, TensorFlow / Face Recognition
- Browser: Google Chrome, Firefox, or Edge

These technologies allow the system to perform facial recognition tasks efficiently while maintaining system stability. They also support scalability and future enhancements.

7.3 Deployment Architecture

The deployment architecture consists of several components that work together to deliver the system functionality. The main components include:

1. Client Interface – This is the user interface where users can register, log in, upload images, and search for missing persons.
2. Web Server – The web server handles user requests and communicates with the backend system.
3. Application Server – This component runs the image processing algorithms and facial recognition models.
4. Database Server – The database stores all missing person records, images, and extracted facial features.
5. AI Model – The trained facial recognition model processes images and compares them with stored records.

These components interact with each other to ensure smooth operation of the system. This architecture supports efficient data flow and system scalability.

7.4 Deployment Process

The deployment process of the missing person identification system involves several steps:

1. Server Setup

The server environment is configured with the required operating system, programming language, and software libraries.

2. Database Configuration

The database is created and structured to store missing person records, user details, and image datasets.

3. Application Installation

The developed application code is uploaded to the server and configured to run the web interface.

4. AI Model Integration

The facial recognition model is integrated into the application so that it can process uploaded images.

5. Testing in Deployment Environment

The system is tested to ensure that all modules work correctly in the real deployment environment.

6. User Access

Finally, the system is made accessible to users through a web browser or online platform.

These steps ensure a smooth transition from development to live usage. Proper deployment reduces errors and improves system reliability.

7.5 Security Considerations

Security is an important factor in the deployment of the system. Since the system stores personal information and images of missing persons, appropriate security measures are implemented.

Some of the security measures include:

- Secure user authentication using login credentials
- Encryption of sensitive data
- Restricted access to the database
- Secure communication between server and client

These measures help protect the data and prevent unauthorized access. Regular security checks are also necessary to handle new threats.

7.6 Maintenance and Updates

After deployment, the system requires regular maintenance to ensure smooth operation.

Maintenance activities include:

- Updating the facial recognition model for better accuracy
- Fixing bugs or system errors
- Adding new features and improvements
- Monitoring system performance

Regular updates help improve the reliability and efficiency of the missing person identification system. Continuous monitoring ensures long-term system stability.

7.7 Summary

In this chapter, the deployment process of the missing person identification system was discussed. The system was successfully deployed in a web-based environment with proper hardware and software configurations. The deployment architecture ensures efficient communication between the user interface, application server, and database.

With proper security measures and maintenance, the system can effectively assist in identifying missing persons using facial recognition technology. It also ensures scalability and readiness for future enhancements.

CHAPTER 8

CONCLUSION AND FUTURE WORK

The proposed system presents an AI-enabled framework for missing person identification that combines MTCNN-based facial detection, deep feature extraction, and a centralized database with community-driven participation. By automating the pipeline from image upload to similarity-based matching and report generation, the system reduces manual effort and improves the speed and accuracy of locating missing individuals in real-world conditions such as variations in lighting, pose, aging, and partial occlusions. Additional features including gender and location filtering, multilingual support, integrated chat, and automated messaging enhance usability and coordination among families, law enforcement agencies, NGOs, and public users. Empirical results from 2022–2024 show a significant increase in case resolution rates after AI integration and NGO-supported reward mechanisms, demonstrating that the framework can positively impact public safety and social welfare outcomes. Overall, the work establishes a scalable and socially responsible platform that bridges advanced technology with humanitarian objectives in missing person recovery.

Future work can focus on improving robustness, fairness, and scalability of the system. Technically, the facial recognition pipeline can be enhanced using more advanced deep learning architectures, age-progression models, and multi-biometric fusion (combining face with gait, voice, or other cues) to handle severe appearance changes and challenging image conditions. Large-scale deployment will require optimisation of database architectures and indexing strategies to support rapidly growing data volumes while maintaining low response times. On the ethical and privacy side, future extensions should include stronger bias-mitigation techniques, improved anonymisation, strict compliance with data-protection regulations, and transparent consent mechanisms for image use. The framework can also be expanded to cross-regional and cross-national interoperability, integrating with external law-enforcement and NGO platforms to support broader, collaborative missing-person search operations

CHAPTER 9

REFERENCES

1. M. K. Singh, P. Verma, A. S. Singh, and K. Anandhan," Implementation of Machine Learning and KNN Algorithm for Finding Missing Person," 2022 2nd International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE), pp. 28-29, 2022.
2. C. Geetha, V. Leelavathi, R. Meharunissa, and V. Nivedita, "FDR: An Automated System for Finding Missing People," 2022 2nd International Conference on Technological Advancements in Computational Sciences (ICTACS), pp. 10-12, 2022.
3. S. S. Kim, H. T. Jung, S. J. Lee, J. H. Park, S. H. Yu, and J. H. Go," A Study of Real-Time 4K Drone Images Visualization to Rescue for Missing People," 2022 13th International Conference on Information and Communication Technology Convergence (ICTC), pp. 19-21, 2022.
4. M. K. Singh, P. Verma, A. S. Singh, and K. Anandhan," IoT Smart Tracking Device for Missing Person Finder," 2022 International Conference on Emerging Trends in Engineering and Technology (ICETET), pp. 58-60, 2022.
5. P. Verma, K. Anandhan, and M. Kumar," Finding Missing Person Using AI," 2022 International Conference on Computational Intelligence and Networks (CINE), pp. 102-105, 2022.
6. S. Kumar, H. Verma, and M. K. Singh, "AI-Assisted Search for Missing Children," International Journal of Computer Vision and Applications, vol. 34, no. 3, pp. 245-259, 2024.
7. R. Meharunissa, V. Leelavathi, and C. Geetha," Finding Missing Person Using Face Recognition," 2022 IEEE International Conference on Computer Vision and Pattern Recognition, pp. 112-115, 2022.
8. P. Verma, M. Kumar, and A. K. Singh," Integration of IoT for Tracking Missing Persons," IEEE Internet of Things Journal, vol. 11, no. 6, pp. 645-659, 2023.
9. S. S. Kim, S. J. Lee, H. T. Jung, and S. H. Yu," AI Camera and IoT Integration for Missing Person Search," IEEE Transactions on Internet of Things, vol. 9, no. 7, pp. 715-720, 2023.
10. M. K. Singh, H. Kumar, and P. Verma, "4K Drones for Real-Time Search Operations," IEEE Transactions on Automation Science and Engineering, vol. 19, no. 8, pp. 535-542, 2022.

APPENDIX

```
import cv2
import numpy as np
from mtcnn import MTCNN
from keras_facenet import FaceNet
from sklearn.metrics.pairwise import cosine_similarity
import os

detector = MTCNN()
embedder = FaceNet()

def detect_face(image):

    faces = detector.detect_faces(image)
    if len(faces) > 0:

        x, y, w, h = faces[0]['box']
        face = image[y:y+h, x:x+w]
        face = cv2.resize(face,
(160,160))
        return face
    return None

def get_embedding(face):
    face = face.astype('float32')
    face = np.expand_dims(face, axis=0)
    embedding = embedder.embeddings(face)
    return embedding[0]

known_embeddings = []
known_names = []

dataset_path = "dataset"

for person in os.listdir(dataset_path):

    person_folder = os.path.join(dataset_path, person)

    for image_name in os.listdir(person_folder):

        image_path = os.path.join(person_folder, image_name)
        image = cv2.imread(image_path)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        face = detect_face(image)

        if face is not None:

            embedding = get_embedding(face)
            known_embeddings.append(embedding)
            known_names.append(person)

print("Known faces loaded:", len(known_embeddings))
```

```

cap = cv2.VideoCapture(0)

while True:

    ret, frame = cap.read()    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    faces = detector.detect_faces(rgb_frame)

    for face in faces:

        x, y, w, h = face['box']    detected_face = rgb_frame[y:y+h, x:x+w]

        if detected_face.size == 0:

            continue

        detected_face = cv2.resize(detected_face, (160,160))    embedding =
get_embedding(detected_face)

        similarities = cosine_similarity([embedding], known_embeddings)    best_match_index =
np.argmax(similarities)    best_score = similarities[0][best_match_index]

        name = "Unknown"

        if best_score > 0.6:

            name = known_names[best_match_index]

            cv2.rectangle(frame,(x,y),(x+w,y+h),(0,255,0),2)    cv2.putText(frame,name,(x,y-
10),cv2.FONT_HERSHEY_SIMPLEX,0.9,(0,255,0),2)

            cv2.imshow("Face Recognition", frame)

            if cv2.waitKey(1) & 0xFF == 27:

                break

cap.release()

cv2.destroyAllWindows()

```